内核设计优化实现文档

BuildStorm 场景下 MossOS 全链路优化

参赛院校： 重庆邮电大学
参赛队名： 每天睡满八小时
队伍成员： 吴勋、张可欣
指导教师： 马彬、钟福金

2026 年 **8** 月

目录

1 概述 1
2 问题定位与根因分析 2
2.1 负载特征 2
2.2 火焰图定位问题 2
2.3 与 Linux 基线的同口径对比 3
2.4 根因分析 4
3 优化设计与实现 8
3.1 优化一：per-CPU 物理页缓存（PCP） 8
3.2 优化二：mprotect 按 VMA 定位并批量修改权限 9
3.3 优化三：AddrSpace 三段式 ——“prepare/execute/commit” 11
3.4 优化四：Folio 系列：folio 页缓存与只读文件页直接映射 14
3.5 优化五：ext4 锁外数据读写 + VirtIO 多在途请求 17
3.6 优化六：地址空间回收与 TLB 刷新 20
3.7 其他优化 22
4 实验分析 24
4.1 实验环境与方法 24
4.2 性能数据展示 24
4.3 LoongArch 火焰图对比 28
4.4 小结 29
5 AI 使用说明与可复现步骤 31
5.1 AI 使用说明 31
5.2 可复现步骤 31
6 总结与展望 33

第 1 章 概述

BuildStorm 题目要求内核在 QEMU 模拟的 RISC-V64 与 LoongArch64 平台上离线从源码全量编译 `arceos-helloworld`，这是一次数百 crate 的真实 Rust 编译，伴随重度文件读写、多线程与大量 `mmap`/`mprotect` 系统调用。评分上，编译成功为门槛，加速分按编译耗时与同一评测机 Linux 基线 `B` 的关系计算，因此能否完整跑通并显著快于两倍 Linux 基线直接决定本题得分。

针对这一目标，我们首先在带 guest profiler 的内核上完整运行 BuildStorm，通过火焰图与路径探针定位问题与根因。结果显示瓶颈集中在文件缺页、页缓存与文件系统 I/O 链路，主要问题包括：

- 缺页处理持有地址空间独占锁等待磁盘 I/O；
- 物理页分配与释放集中在全局分配器；
- 只读文件映射多做一次私有页分配与复制；
- 页缓存按单页组织，逐页查找、重复读盘；
- ext4 全局锁内等待块设备；
- `mprotect` 反复扫描全局映射记录。

这些全局锁、逐页处理和重复工作使 8 个 vCPU 的并行能力无法发挥。本次 LoongArch 三次重测中，优化前基线的 6～8 核时间占比仅 0.20%。围绕上述根因，我们完成了以下关键优化：

- per-CPU 物理页缓存（PCP），缓解全局分配器竞争；
- per-VMA `mprotect`，只更新目标区间；
- AddrSpace 三段式缺页（prepare / execute / commit），慢操作移出地址空间锁；
- folio 页缓存，合并相邻页的加载与 I/O；
- 只读文件页直接映射，真正写入时才 COW；
- ext4 数据 I/O 移出全局锁，配合异步 VirtIO 多在途请求；
- 后续补充：VFS 目录与 inode 元数据并发、缺页提交微路径、异步地址空间回收与 TLB 合并刷新、精确 TLB shootdown。

实验数据均为本地 LoongArch64 重测所得（QEMU 10.0.2 `virt`、8 vCPU、8 GiB、同一磁盘镜像、关闭 profiler）。Linux 基线和 Moss 十个阶段均串行运行三次，以评分脚本输出的有效编译时长中位数作为报告值。在此口径下，Moss 从 993.08 秒降至 354.27 秒，减少 64.33%，加速 2.80 倍。最终平均有效核数由 3.077 核提高到 3.769 核，6～8 核时间占比由 0.20% 提高到 35.98%。

目前，Moss 内核已在评测机上完成 BuildStorm 测例的正式测评，RISC-V64 与 LoongArch64 两侧得分均为 **180** 分。排行榜截图如下：

#	用户名	队伍	学校	提交次数 (ASC)	buildstorm-glibc-la	buildstorm-glibc-rv	cagent-glibc-la	cagent-glibc-rv	rank
1	T2026104879910415	维塔立体	华中科技大学	46	180.0	180.0	199.1	199.1	758.2000
2	T202610617999569	每天睡满八小时	重庆邮电大学	60	180.0	180.0	199.1	199.1	758.2000
3	T202612332999758	这行代码有问题	烟台南山学院	76	180.0	180.0	199.1	199.1	758.2000
4	T2026104869910383	洗剪吹	武汉大学	77	180.0	169.3	199.1	199.1	747.5000
5	T202610459999616	PulseOS	郑州大学	19	180.0	151.3	199.1	199.1	729.5000

图 1　BuildStorm 测评成绩截图（截至 2026.8.12）

第 2 章 问题定位与根因分析

2.1 负载特征

BuildStorm 在计时前进行工具链与 minibuild 校验，预编译部分不计入成绩；正式计时窗口仅包含 `cargo xtask arceos build -p arceos-helloworld`。该命令内部仍需从源码编译目标及其依赖。从内核视角看，这一负载有以下特征：

- 文件访问密集：正式编译会读取大量 crate 源码和依赖元数据，并生成 `.rlib`、对象文件及临时产物；rustc/LLVM 会通过 `mmap(MAP_PRIVATE)` 访问其中大量文件，从而产生密集的文件缺页和页缓存访问；
- 多线程高并发：cargo/rustc 并行调度，8 个 vCPU 上同时存在多个编译任务与页缺失；
- 进程生命周期短：rustc、链接器、构建脚本频繁 fork/exec/exit，地址空间反复建立与销毁，`exec` 需要整体切换地址空间；
- 内存映射系统调用频繁：`mmap/munmap/mprotect` 被大量调用，其中 `mprotect` 尤其密集；
- 文件系统 **I/O** 重：冷缓存时大量页缓存 miss 需要进入 ext4 查询块映射并等待块设备。

这些特征叠加在一起，使得“单次系统调用本身的耗时”并不重要，重要的是缺页、页缓存、文件系统与内存管理路径上的全局锁竞争和串行等待被放大。

2.2 火焰图定位问题

火焰图与正式计时分开采集，避免探针扰动混入性能结果。2026-08-13 在与正式实验相同的 LoongArch64 环境中补跑了两类 profiling：CPU 图通过 QEMU gdbstub 每 2 秒暂停一次并采集 8 个 vCPU 的调用栈；等待图只覆盖赛题正式计时窗口，阻塞式 mutex 的每次实际睡眠等待都记录调用栈和持续时间，`mprotect` 因调用频繁而每 32 次采样一次并按采样率折算累计时间。环境统一为 QEMU 10.0.2、8 vCPU、8 GiB 和同一镜像。等待插桩阶段分别为基线 `stage-00`、优化前的 `stage-03`、合入 `mprotect` 优化的 `stage-04` 和最终版本 `stage-09`；所有 profiling 均为独立运行，不计入第 4 章正式耗时。

赛题只计时 `cargo xtask arceos build -p arceos-helloworld`（即实验中的 timed 阶段，`elapsed_s`），因此等待探针只在该窗口开启。下图是优化前基线独立采集的 CPU 火焰图：

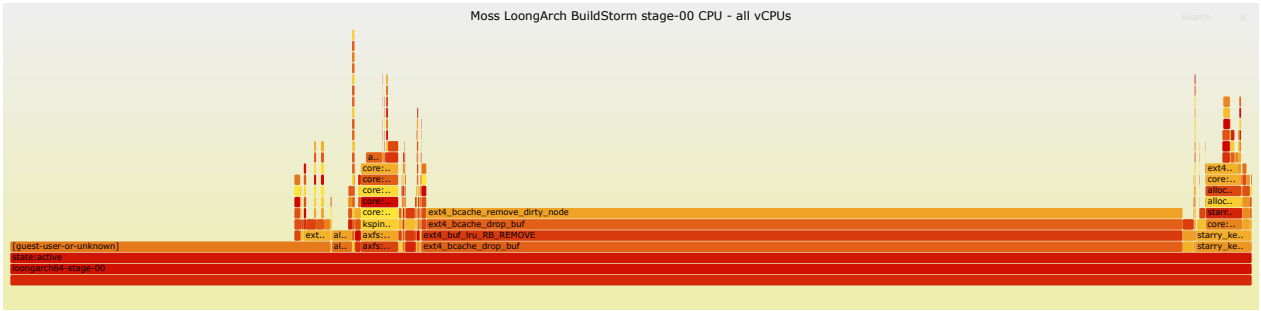


图 2　优化前 LoongArch CPU 火焰图（4112 个 vCPU 样本）

本轮共取得 4112 个 vCPU 样本，其中 3171 个解析为内核符号。图中最突出的现象不是某个计算函数耗时很长，而是 ext4 buffer-cache 清理形成了高度集中的热塔：`ext4_bcache_drop_buf` 覆盖 80.48% 的已解析内核样本，`ext4_bcache_remove_dirty_node` 作为叶节点占全部样本的 60.92%。这说明大量 CPU 时间消耗在文件系统缓冲区维护，而没有直接推进编译；其余热点明显更分散。

CPU 图中的 gdb 栈不能可靠识别 guest 用户态与 idle，因此不从这张图计算并行度；并行度使用第 2.3 节宿主机逐秒采样。主要叶节点列于下表：

热点	样本数	占比
ext4_bcache_remove_dirty_node	2505	60.92%
guest user / 未解析	941	22.88%
LruCache<PageCache>::put	68	1.65%
ext4_bcache_init_dynamic	55	1.34%
FsContext::lookup	54	1.31%
BTreeMap VMA unmap	54	1.31%
alloc::alloc::dealloc	39	0.95%
atomic_or<u8>	35	0.85%

表 2-1　优化前 LoongArch CPU 叶节点热点（占全部 4112 个 vCPU 样本）

LoongArch 基线的 mutex 等待火焰图如下。该轮完整记录了 111602 次实际睡眠等待，累计 2401.96 秒；其中 `BackendOps::populate -> CowBackend::alloc_new_at` 文件缺页路径占 84.92%，形成图中最宽的塔。由于多个任务的等待可以同时发生，累计等待会大于 1045.06 秒的插桩运行墙钟时间。结果说明大量编译任务并不是没有被调度，而是在同一条 COW/文件缺页链上层层排队。

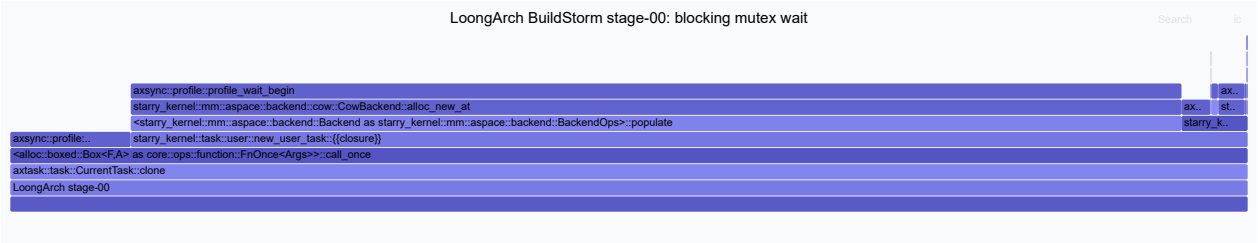


图 3　优化前 timed 阶段 mutex 等待火焰图

开发阶段更细的 RISC-V 路径探针还给出了页缓存与文件系统内部的时间去向；它与本次 LoongArch mutex 图用于不同层次的证据，数值不作跨架构直接比较：

事件	累计值
page-cache inclusive	4042.25 s
mutex wait	3369.74 s
ext4 inclusive	453.85 s
精确 ext4 全局锁等待	103.89 s
同步块读取	170.38 s
同步块写入	6.23 s
all off-CPU	70132.78 s

表 2-2　优化前 timed 阶段路径累计值（跨任务累计，可大于墙钟）

其中 `page-cache inclusive` 占据主导（单个 rustc 缺页栈占 70.82%，各任务合计约 84%），最宽路径仍是缺页；ext4 全局锁等待 103.89 s 中最大的单一路径同样是缺页读取（79.16%）。开发阶段的 CPU/路径探针与本次 LoongArch mutex 图独立指向同一条热点链：”缺页 → `AddrSpace` 大锁 → 页缓存 → ext4 → 块设备”，具体性能热点在下一节逐项展开。

2.3　与 Linux 基线的同口径对比

为避免把旧架构、旧计时边界的探针结果与当前实验混在一起，这里只使用 2026-08-13 的 LoongArch 正式三次中位数。Linux 与 Moss 使用同一 QEMU 10.0.2、8 vCPU、8 GiB 和磁盘镜像；两者都关闭 profiler，并以 `BUILDSTORM_COMPILE` 的有效窗口计时。

指标	**Linux**	**Moss**（优化前）	**Moss / Linux**
有效编译时长	413.31 s	993.08 s	2.40 倍
平均有效核数	4.432	3.077	0.69 倍
6～8 核时间占比	46.79%	0.20%	约 1 / 233
累计 CPU 时间	1842 CPU·s	3069 CPU·s	1.67 倍
平均 RSS	2332 MiB	1864 MiB	0.80 倍

表 2-3　Moss（优化前）与 Linux 的 BuildStorm 同口径对比

优化前 Moss 的有效编译时长是 Linux 的 2.40 倍，平均有效核数反而只有 Linux 的 69%，6～8 核区间几乎没有出现；与此同时，Moss 累计消耗了更多 CPU 时间。这说明差距既不是“给虚拟机更多内存”能够解决的，也不是单纯少做了并行调度，而是大量 CPU 工作和等待没有转化为编译进度。结合前一节 LoongArch mutex 图中 84.92% 的等待集中在 COW 文件缺页路径，首要修复方向就是拆除“AddrSpace → CowBackend → page-cache → ext4 → block”的跨层大临界区。

2.4　根因分析

在逐条分析热点之前，下图展示了优化前发生缺页的完整处理链路。图中左右两侧四根彩色竖线分别标注四种锁的持锁周期——AddrSpace 独占锁、cache_op + page_cache 锁、ext4 全局自旋锁、全局页分配器锁。

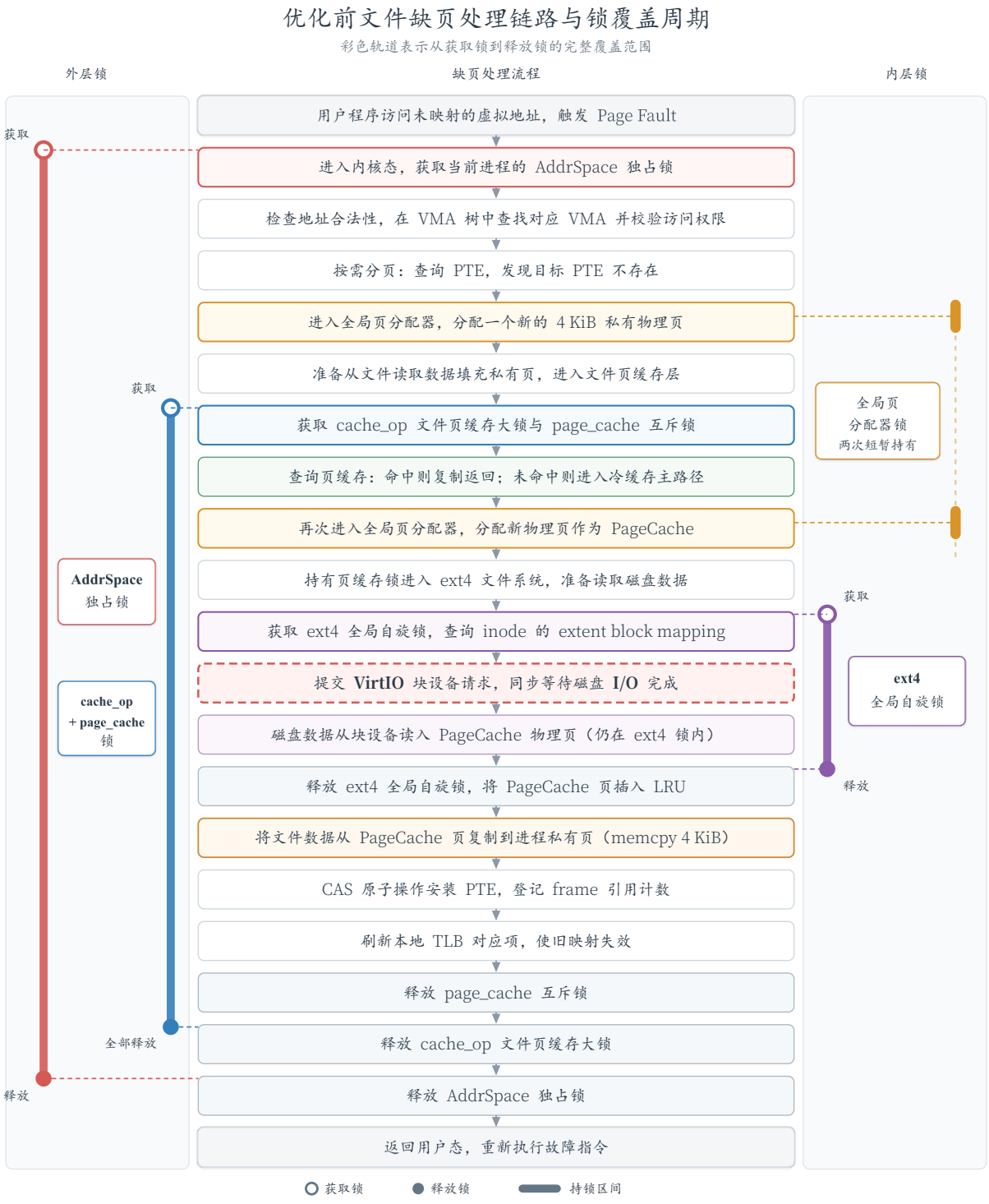


图 4　优化前文件缺页处理链路与锁范围

从图中就可以清晰地看到旧链路的两类结构性问题。

一方面是“大锁套小锁”：外层的 AddrSpace 锁尚未释放就逐层获取 page_cache 锁和 ext4 锁，内层锁上的任何阻塞都会直接穿透到外层的所有等待者。当一个线程在 ext4 锁内等磁盘 I/O 的时候，不仅持有 ext4 锁不放，还同时卡住了 page_cache 锁和 AddrSpace 锁，其他 7 个核上所有需要这些锁的线程就全部都要排队。

另一方面是“锁粒度太大”：同一把锁将快操作和慢操作捆在一起——ext4 锁内既做微秒级的 extent 映射查询，也做毫秒级的同步磁盘等待；AddrSpace 锁内既有轻量的 VMA 查找，也有重量级的物理页分配和文件数据复制。一些不需要锁内进行的重任务也被纳入了锁内。

并且两类问题会互相放大：内层锁做着毫秒级的慢活，外层锁只能被动等待，而 BuildStorm 会同时运行成百上千个 rustc/cc1 进程、每个都在反复缺页读文件。2026-08-13 的 LoongArch 补测中，基线 mutex 累计等待为 2401.96 秒，其中 84.92% 经过图中所绘的 COW 文件缺页路径；正式三次样本的平均有效核数只有 3.077 核，说明 8 个 vCPU 被这条锁链显著串行化。

因此，后面将介绍的六项核心优化的本质方向是一致的：缩短持锁时间（三段式、ext4 锁外 I/O）、降低进锁频率（PCP、folio 合并）、缩小锁的管辖范围（per-VMA mprotect），每一项都在拆除这条锁链中的一环。

结合火焰图、探针累计值与代码阅读，将主要瓶颈归纳为以下六个根因：

2.4.1 根因一：缺页处理持有整个地址空间的独占锁等待慢操作

在旧链路中，一次文件缺页会独占当前进程的 `AddrSpace`，在锁内完成 VMA 查找、物理页分配、页缓存 miss、ext4 块映射查询、磁盘 I/O 等待、数据复制与 PTE 安装。磁盘 I/O 的等待时间远大于 VMA 查询本身；一个线程等盘时，其他线程即使缺的是完全不同的文件或地址，也只能排队。BuildStorm 线程多、文件多，把这种串行等待放大是最主要的性能瓶颈。LoongArch 基线火焰图中，`BackendOps::populate → CowBackend::alloc_new_at` 这条 COW 文件缺页路径占 mutex 累计等待的 84.92%（2039.81 / 2401.96 秒）。

2.4.2 根因二：物理页分配与释放集中在全局分配器

缺页分配私有页、页缓存分配页、COW 分配页以及进程退出时释放页，都会进入全局物理页分配器并竞争同一把锁。8 个 vCPU 同时缺页时，所有 CPU 的分配和释放都被串行化。本次 LoongArch CPU 图的最宽热点被 ext4 buffer-cache 清理占据，无法从中单独量化分配器竞争；全局分配器的代码路径以及合入 PCP 后 `stage-02 → stage-03` 的正式耗时从 443.20 秒降至 426.87 秒，为这一判断提供了相互补充的证据。

2.4.3 根因三：MAP_PRIVATE 只读缺页多一次私有页分配与复制

对于只读使用的 `MAP_PRIVATE` 文件映射，旧实现即使只是读取，也会”先分配页缓存页、读盘、再为进程分配一张私有物理页、从页缓存复制 4 KiB、安装私有页 PTE”。大量只读文件页永远不会被写，私有页分配和 4 KiB 复制完全浪费，且每个进程映射同一文件都会各复制一份，加剧内存压力与缺页成本。这一问题主要来自代码逻辑分析——LoongArch mutex 图中 `CowBackend::alloc_new_at` 是最宽的等待塔，而其中的 memcpy 和私有页分配正是该路径的组成部分，但等待图本身无法直接区分”必要的复制”与”浪费的复制”。

2.4.4 根因四：页缓存粒度小，逐页查找、重复 I/O 与重复等待

旧实现页缓存按单页组织：一次 miss 只加载一张 4 KiB 页，相邻页需要再次进入页缓存查询并可能再次读盘；多个线程同时缺同一页时还可能重复发起 I/O。开发阶段的 RISC-V 路径探针中，page-cache inclusive 累计 9343.23 秒，是量级最大的嵌套路径，火焰图中占据约 91.5% 的图宽，其中 71.85% 经过 `with_page_or_insert_impl`。这组旧数据只用于说明定位过程，不与本次 LoongArch 结果作数值比较。

2.4.5 根因五：ext4 全局锁内等待块设备

Moss 用一把全局锁保护 `LwExt4Filesystem` 元数据。旧实现中一次文件读取在锁内完成块映射查询后继续等待磁盘 I/O 完成才释放锁，导致不同文件的数据读取被全局串行化。开发阶段的 RISC-V 探针中，ext4 全局锁等待累计 218.92 秒、同步块读取 175.67 秒，最大的单一调用栈同样是 rustc 文件映射缺页（占 64.80%）。本次 LoongArch CPU 图又显示 `ext4_bcache_drop_buf` 覆盖 80.48% 的已解析内核样本，说明文件系统内部串行与缓冲区维护仍是基线的核心问题。

2.4.6 根因六：mprotect 扫描全局映射记录与无关 VMA

`mprotect` 并不是一开始最显眼的瓶颈。最初采集火焰图时，文件缺页、页缓存和 ext4 I/O 形成的宽大热塔占据了绝大部分采样，`mprotect` 这种调用频繁但单次耗时相对较短的路径被压在下面，很

难单独看出来。随着前几项优化逐步拆开缺页锁链、缩短页缓存和文件系统路径，原来的主热点明显收窄；再次观察火焰图时，性能瓶颈已经发生转移，`mprotect` 开始成为不可忽略的高频热点。

沿着这条新出现的热点继续检查，可以看到旧实现维护了一个由所有进程共享的 `MMAP_RECORDS` 数组。每次 `mprotect` 都要先锁住这个全局数组，遍历其中的映射记录以检查 seal 和最大权限；修改页表后，还要再次获取同一把锁并重建记录数组。与此同时，`MemorySet::protect` 会从地址空间头部遍历 VMA，即使调用实际只覆盖很小的一段地址，也会检查大量无关映射。随着并发进程和映射数量增加，这些全局扫描、重复加锁和逐页 TLB 刷新不断累积，原本很短的一次权限修改就被放大成新的串行开销。

路径采样进一步印证了这一判断：优化前约 6720 次 `mprotect` 调用的折算累计延迟为 1.907 秒，单个采样区间最大达到 9.74 毫秒。优化后调用数量基本没有变化，累计延迟却降至 0.468 秒，下降 75.45%，最大采样延迟也降至 0.96 毫秒。这说明性能改善并不是因为编译器减少了 `mprotect` 调用，而是按 VMA 定位、局部更新权限和批量刷新 TLB 真正缩短了每次调用的内核路径。

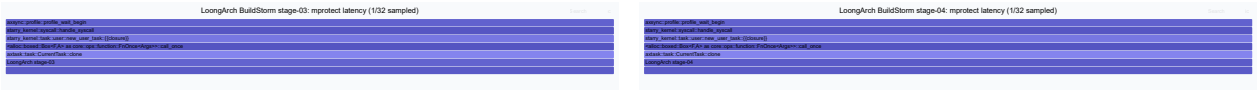


图 5　`mprotect` 延迟火焰图对比（左：优化前；右：优化后）

第 3 章 优化设计与实现

本章按“六项核心优化 + 其他优化”两部分组织，六项核心优化按依赖顺序实施，其余针对次生热点的改进归入“其他优化”。每项优化均遵循同样的工程约束：修改独立提交、保留原始测量数据、先通过正确性测试再评估性能，避免把多项修改打包后无法归因。实现提交与实验数据的对应关系见第 4 章。

3.1 优化一：per-CPU 物理页缓存（PCP）

3.1.1 设计思路

如 2.4 根因所述，所有 order-0（4 KiB）物理页的分配与释放都集中在全局分配器，8 个 vCPU 并发缺页时会竞争同一把全局锁。本次 LoongArch CPU 图主要被 ext4 buffer-cache 热点覆盖，因此不再沿用旧 RISC-V 火焰图对分配器热点的占比；PCP 的效果改由相邻阶段正式耗时与实现路径共同判断。

为此在每个 CPU 身边维护一小撮空闲 4 KiB 物理页（PCP, Per CPU Pagecache），分配和释放优先命中本地缓存，仅在本地未命中或缓存空/满时才批量访问全局分配器。进行物理页分配时，若 PCP 缓存中没有，则批量从全局分配器补充，放入 PCP；释放物理页时，先放回 PCP，缓存过多时再批量归还全局分配器。这样可以把原本逐页发生的全局访问合并为少量批量操作，显著减少访问全局分配器锁的次数，直接缓解多核并发下的锁竞争，因此能够有效提升 BuildStorm 的性能。

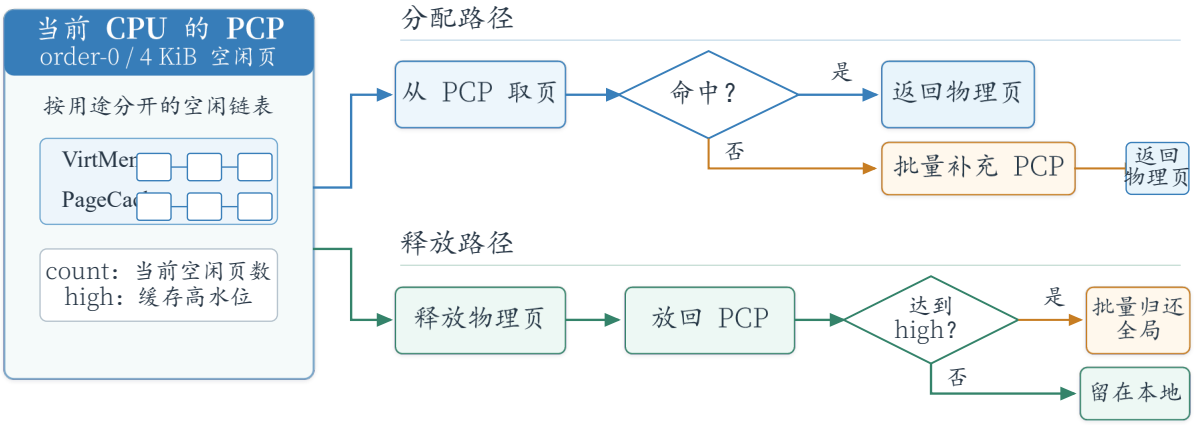


图 6　PCP 物理页缓存：分配与释放流程

3.1.2 实现要点

核心数据结构 `PerCpuPageCache` 为每种 `UsageKind` 维护一条独立空闲链表，通过 `cpu_id % 64` 定位槽位，`batch` 和 `high` 两个参数控制批量进出粒度：

```
pub(crate) struct PerCpuPageCache {
    // 每种 UsageKind（VirtMem / PageCache / ...）一条空闲链表
    heads: [usize; UsageKind::VARIANTS.len()],
    kind_counts: [usize; UsageKind::VARIANTS.len()],
    count: usize,           // 当前缓存总页数
    batch: usize,           // 每次 refill / drain 的批量页数
    high: usize,            // 当前高水位线
    high_min: usize,        // 高水位下限（8 GiB / 8核 ≈ 624）
    high_max: usize,        // 高水位上限（≈ 32768）
    // ...
}
```

分配路径：快路径仅持本 CPU 槽位的 IRQ 锁，命中时直接从链表头部弹出；未命中再走慢路径加全局分配器锁批量补货。

```
pub fn alloc_page_cached(&self, cpu_id: usize, cpu_count: usize,
                         kind: UsageKind) -> AllocResult<usize> {
    let slot = cpu_id % PCP_CPU_SLOTS;
    // 快路径：只锁本 CPU 槽位，IRQ 级别即可
    let mut cache = self.page_caches[slot].lock();
    if let Some(page) = cache.try_allocate(total_pages, cpu_count, kind) {
        return Ok(page);   // 命中，直接返回，全程不碰全局锁
    }
    drop(cache);
    // 慢路径：关抢占 + 全局分配器锁，批量 refill batch 页
    self.alloc_page_cached_slow(slot, total_pages, cpu_count, kind)
}
```

释放路径：先推回本 CPU 的 PCP；当缓存页数超过高水位时，批量退还 batch 页给全局分配器，避免空闲页在 PCP 中积压导致其他 CPU 或高阶分配无法获取。

```
pub fn dealloc_page_cached(&self, cpu_id: usize, cpu_count: usize,
                           page: usize, kind: UsageKind) {
    let slot = cpu_id % PCP_CPU_SLOTS;
    let _preempt_guard = NoPreemptIrqSave::new();  // 关闭抢占
    let mut cache = self.page_caches[slot].lock();
    cache.release(self, total_pages, cpu_count, page, kind);
    // release 内部：push 入链表；若 count >= high，调用 drain 批量退页
}
```

全局分配失败（如剩余内存不足或高阶连续页无法满足）时，慢路径会先调用 `drain_page_caches()` 把所有 CPU 的 PCP 缓存全部退回全局分配器再重试一次，既防止空闲页被藏在 PCP 中造成的“假 OOM”，也为高阶连续页合并创造了机会。

3.2 优化二：mprotect 按 VMA 定位并批量修改权限

`mprotect(addr, len, prot)` 的作用是修改当前进程 `[addr, addr+len)` 这段虚拟地址的访问权限，例如将一段内存从可读写改为只读。BuildStorm 中 rustc/LLVM 会频繁调用 `mprotect` 来翻转代码段和数据段的权限，调用量极大（预构建约 147 万次、正式阶段约 65 万次）。

旧实现维护了一个全局的 `MMAP_RECORDS` 数组，混合保存所有进程的映射记录。一次 `mprotect` 需要锁住该全局数组，扫描全部记录以检查 seal 和最大权限，修改页表后再次锁住并重建整个数组。平均一次调用扫描约 3.16 万条记录，与当前进程实际涉及的区间大小完全无关。新旧链路对比如下：

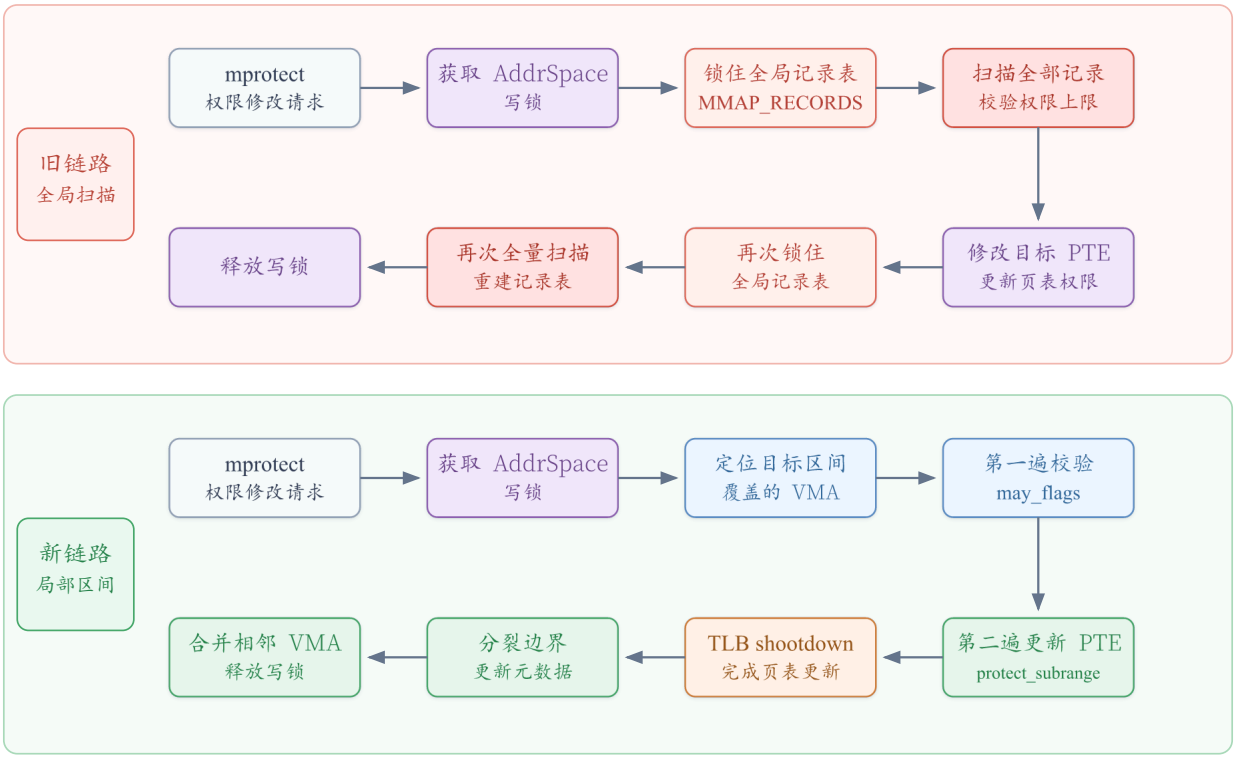


图 7　mprotect 按 VMA 定位：旧全局扫描 vs 新局部区间

优化的核心思路是删除全局 `MMAP_RECORDS`，把权限元数据直接嵌入每个 VMA。每个 `MemoryArea` 携带一个 `VmaMetadata`，其中 `may_flags` 记录了该 VMA 允许的最高权限——`mprotect` 只需检查目标区间的几个 VMA 是否在 `may_flags` 允许范围内即可，不用再扫描全局记录。

```
/// 嵌入每个 VMA 的权限元数据，替代全局 MMAP_RECORDS
pub struct VmaMetadata {
    user_flags: MappingFlags,  // 用户当前看到的权限
    may_flags: MappingFlags,   // 允许提升到的最大权限（seal / 只读文件限制）
    shared: bool,
    file_backed: bool,
    anonymous: bool,
    locked: bool,
}
```

`mprotect` 必须保证“要么全部成功，要么全部失败”，因此采用两遍式。第一遍只校验不修改：cursor 沿 `[addr, addr+len)` 前进，每次用 `range(..=cursor).next_back()` 定位覆盖 cursor 的最后一个 VMA，检查 `may_flags` 是否允许目标权限；若目标权限与当前一致则标记 no-op。复杂度 `O(log N + K)`，仅访问实际覆盖的 K 个 VMA。

第二遍才实际修改：对每个受影响的 VMA，调用 `protect_subrange` 更新相交区间的 PTE 权限（仅修改实际需要变化的子范围），必要时按左右边界分裂 VMA，更新目标段 flags/元数据后，将兼容的相邻 VMA 合并，避免了地址空间碎片化。

```
// MemorySet::protect —— 两遍式 VMA 区间修改
fn protect(&mut self, start, size, update_flags, can_merge, pt) -> Result {
    // 第一遍：纯校验，不改任何数据
    let mut cursor = start;
    while cursor < end {
        let area = self.find(cursor)?;          // O(log N) 定位
```

```
        if update_flags(area)?.is_some() { needs_update = true; }
        cursor = area.end().min(end);
    }
    if !needs_update { return Ok(no_op); }      // 权限已一致，直接返回

    // 第二遍：修改 PTE → 分裂 VMA → 更新元数据 → 合并相邻 VMA
    cursor = start;
    while cursor < end {
        let area = self.find_mut(cursor)?;
        area.protect_subrange(cursor, len, new_flags, pt)?;  // 只改 PTE 子范围
        if area.start() < cursor { split_left(); }           // 按左边界分裂
        if area.end() > end { split_right(); }               // 按右边界分裂
        area.update_flags_and_meta(new_flags);
        merge_if_compatible(&mut self.areas);                // 合并相邻兼容 VMA
    }
}
```

但是还可能出现一种问题：当 mprotect 收紧权限（如可写改成只读）时，修改 PTE 只是第一步，但是其他 CPU 的 TLB 中仍缓存着旧的可写映射，如果此时就释放物理页，远端 CPU 就还可能通过旧 TLB 写入已被回收的页面。 所以多核 TLB 同步方面要遵守一定的释放顺序：

```
先修改 PTE
        -> 向所有相关 CPU 发送 TLB shootdown（RISC-V 通过 SBI RFENCE），确保旧映射在全部核心上失效
        -> 最后才能释放 frame/folio 引用
```

全程持有 `AddrSpace` 写锁，保证 VMA 树与页表不被并发的结构性修改打断。

3.3 优化三：AddrSpace 三段式 —— “prepare/execute/commit”

3.3.1 设计思路

如 2.4 根因所述，缺页在持有整个进程 `AddrSpace` 独占锁期间执行分配、页缓存、ext4 与磁盘 I/O，一个线程等盘时其他缺页、`mmap`、`munmap`、`mprotect` 全部排队（LoongArch 基线 mutex 累计等待 2401.96 秒，其中 84.92% 经 COW 文件缺页路径）。因此我们把缺页处理拆成 prepare / execute / commit 三个阶段，将慢操作全部移到 `AddrSpace` 锁外：



图 8　AddrSpace 三段式缺页处理流程

- **prepare**：持有读锁，只做判断，确定要处理什么——定位 VMA、校验权限、检查 PTE 是否已被其他线程装好、构造 `FaultPreparation`/fault plan，不读盘、不分配；
- **execute**：不持有 `AddrSpace` 锁，并发执行一些慢操作：物理页分配、页缓存查找/创建、等待 folio loader、进入 ext4、等待磁盘、COW 数据复制等；
- **commit**：重新取得读锁，再次定位 VMA 并校验权限，确认 prepare 时的前提仍成立后用原子 CAS 安装 PTE，同时登记 PTE 对应的 frame/folio 所有权。

提交结果分为 `Handled`（本线程成功或目标已被正确处理）、`Stale`（前提过期，需要重试）与错误三类。实现用 `MAX_CONCURRENT_FAULT_RETRIES = 4` 限制乐观重试次数，连续 stale 后回退到拿写锁的旧慢路径，保证极端竞争下仍能推进。

3.3.2 实现要点

prepare 阶段在 `AddrSpace` 读锁内完成判断，返回一个 `FaultPreparation` 枚举值；execute 阶段在锁外执行，完事后在 commit 阶段重新获取读锁、验证前提并原子提交，最终返回 `Handled` 或 `Stale`：

```
/// prepare 的结果：要么已就绪，要么带一个可锁外执行的 fault plan
pub(crate) enum FaultPreparation {
    Handled,                        // PTE 已被其他线程安装，无需处理
    Cow(CowFaultPlan),              // 私有页（匿名/COW 文件）缺页计划
    File(FileFaultPlan),            // 共享文件 folio 缺页计划
    Shared(SharedFaultPlan),        // 共享匿名页缺页计划
    Fallback,                       // 退回旧写锁慢路径
}

/// commit 的结果
pub(crate) enum FaultCommit { Handled, Stale }
```

核心调度逻辑在 `handle_page_fault_concurrent_inner` 中：循环内先拿读锁做 prepare，再在锁外 execute，最后拿读锁 commit；`Stale` 时有限重试，连续失败后退回 `Fallback` 走写锁慢路径：

```
fn handle_page_fault_concurrent_inner(aspace, vaddr, access_flags, ...) -> bool {
    let mut stale_retries = 0;
    loop {
        // ── Phase 1: prepare（持有 AddrSpace 读锁）──────────────
        let preparation = {
            let guard = aspace.read();
            // 校验地址、定位 VMA、检查权限
            area.backend().prepare_fault(vaddr, area_range, flags, access_flags,
&guard.pt)
        };  // 读锁在此释放

        // ── Phase 2: execute（不持有 AddrSpace 锁）──────────────
        match preparation {
            FaultPreparation::Handled => { flush_tlb; return true; }
            FaultPreparation::Fallback => break,  // 退回写锁慢路径
            FaultPreparation::Cow(mut plan) => {
                plan.execute()?;  // 锁外：分配页、读盘、COW 复制
                // ── Phase 3: commit（重新获取读锁）──────────
                let guard = aspace.read();
                // 重新定位 VMA、重新校验权限
                area.backend().commit_fault(&mut plan, ...)
                // commit 内部：CAS 安装 PTE，登记 frame 所有权
            }
            // File / Shared 分支同理...
        };
        match committed {
            FaultCommit::Handled => { flush_tlb; return true; }
            FaultCommit::Stale => { stale_retries += 1; continue; }  // 有限重试
        }
    }
    // Fallback：拿写锁走旧慢路径，保证极端竞争下仍能推进
```

```
    aspace.write().handle_page_fault(vaddr, access_flags)
}
```

prepare 和 commit 都使用读锁而非写锁，原因是 VMA 树的结构性修改（mmap/munmap/改变 VMA 的 mprotect）仍走写侧，而缺页线程只读 VMA、不改变树结构，PTE 的并发安装由更细粒度的原子 CAS 协调。

3.4 优化四：Folio 系列：folio 页缓存与只读文件页直接映射

Folio 系列同时解决两个根因：页缓存粒度小导致的逐页查找与重复 I/O（根因四），以及只读 MAP_PRIVATE 缺页多一次私有页分配与复制（根因三）。

3.4.1 folio 页缓存：批量加载与并发等待

Folio 这个做法参考了 Linux 的部分实现。将文件页缓存的管理单位从单页提升为 folio（一组连续的 4 KiB 基础页，最大 order = 4，即一个 folio 最多覆盖 16 页共 64 KiB）。用户页表仍按 4 KiB 安装 PTE，folio 只改变页缓存的查找、I/O、状态管理与回收粒度。

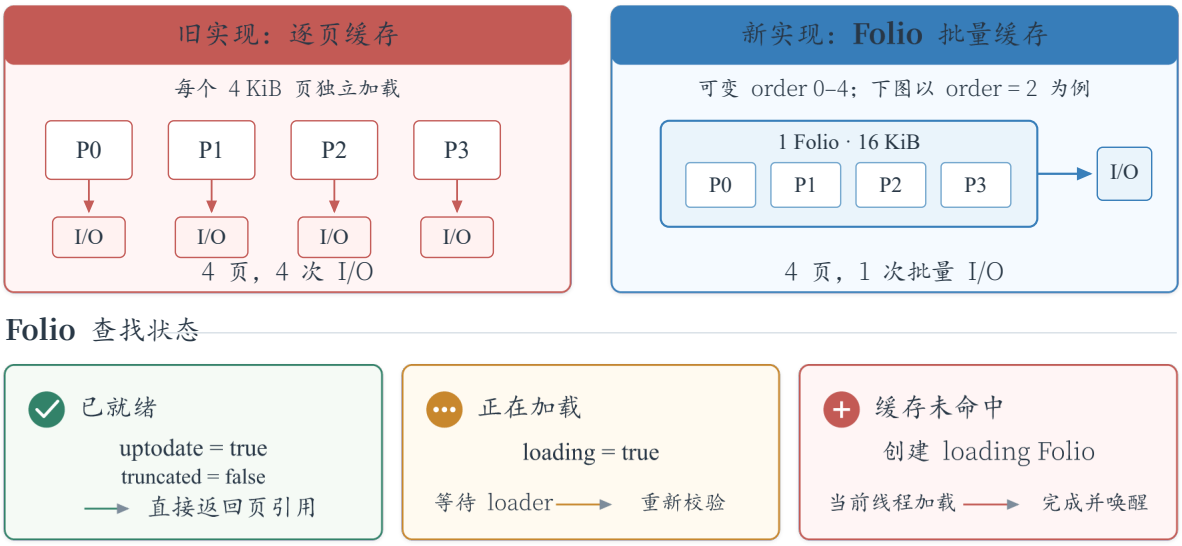


图 9　Folio 页缓存：单页 vs 批量加载，以及查找 Folio 的三种状态

每个 Folio 内部维护一个 FolioState，核心字段为 loading、uptodate、truncated 三个布尔标志，分别表示”正在从磁盘加载”、”数据已有效”、”已被截断失效”。查找 folio 时根据这三个字段走三条路径：已就绪则直接返回引用（最快，仅持每页锁）；正在加载则释放锁后等待 loader 完成再重新校验；缓存未命中则创建新 folio 并置 loading = true，由本线程作为唯一的 loader 发起磁盘 I/O，完成后置 uptodate = true 并唤醒所有等待者。

```
/// 每个 Folio 自身的生命周期状态
struct FolioState {
    loading: bool,          // 正在从磁盘加载数据
    uptodate: bool,         // 数据已有效，可以安全读取
    error: Option<VfsError>, // 加载失败时记录错误原因
    dirty: u16,             // 位图：哪些子页被修改过
    writeback: u16,         // 位图：哪些子页正在回写
    truncated: bool,        // 文件已被截断，缓存失效
}

pub struct Folio {
```

```
    index: u32,             // 起始页号（自然对齐到 order）
    order: u8,              // 2^order 页，最大 4（16 页 = 64 KiB）
    frame: FolioFrame,      // 连续物理页
    state: Mutex<FolioState>,
    page_locks: [Mutex<()>; MAX_FOLIO_PAGES], // 每子页一把锁
    wait: WaitQueue,        // 等待 loading 完成的队列
    // ...
}

// loader 完成 I/O 后调用：更新状态并唤醒所有等待者
fn complete_load(&self, result: VfsResult<()>) {
    let mut state = self.state.lock();
    state.loading = false;                    // 加载结束
    if result.is_ok() && !state.truncated { state.uptodate = true; }
    else { state.error = Some(...); }
    drop(state);
    self.wait.notify_all(false);              // 唤醒所有 waiter
}

// 等待方调用：阻塞直到 loading 变为 false
fn wait_uptodate(&self) -> VfsResult<()> {
    self.wait.wait_until(|| !self.state.lock().loading);
    // 醒来后检查 uptodate / error / truncated
}
```

页缓存索引按文件分成 64 个 shard，不同文件的缺页线程争抢不同 shard 的锁；同一 folio 只允许一个 loader 真正读盘，其他线程找到 `loading` folio 后等待其完成，避免重复读盘。用 `FolioPageRef` 保证已装入 PTE 的物理页在页缓存回收期间不会被提前释放。等待 loading folio 时必须先释放 `invalidate_lock`，完成后再重新加锁并检查同一页号是否仍是同一个 folio（因为等待期间 truncate 可能已替换），避免三方死锁。folio 并非越大越好，保留单页 fallback。

```
/// 每个文件的 folio 索引：64 个 shard 降低锁竞争
struct FileAddressSpace {
    folio_index: [RwLock<BTreeMap<u32, Arc<Folio>>>; 64],
    invalidate_lock: SleepRwLock,  // truncate 等破坏性操作持写侧
    // ...
}

/// 持有 Folio 的强引用，保证 PTE 指向的物理页不被回收
pub struct FolioPageRef {
    folio: Arc<Folio>,  // Arc 引用计数阻止 folio 被释放
    page: u8,           // 子页在 folio 内的偏移
}

/// 查找 folio 页的完整流程：释放锁等待 → 重新加锁 → 校验未变
fn with_page(&self, pn: u32, f: ...) -> R {
    // ① 持 invalidate 读锁，查 folio 索引
    let folio = {
        let _invalidate = self.invalidate_lock.read();
        self.lookup_folio(pn)
```

```
    };
    // ② 释放 invalidate 锁后再等待 loading（避免三方死锁）
    if folio.wait_uptodate().is_err() { return f(None); }
    // ③ 重新获取 invalidate 读锁，校验仍是同一个 folio
    let _invalidate = self.invalidate_lock.read();
    let current = self.lookup_folio(pn);
    if !Arc::ptr_eq(&current, &folio) { return f(None); }  // truncate 已替换
    // ④ 持每页锁 + 校验 uptodate/truncated → 安全访问
    let page_guard = folio.page_locks[page].lock();
    let state = folio.state.lock();
    if !state.uptodate || state.truncated { return f(None); }
    // ...
}
```

3.4.2 只读文件页直接映射（COW）

旧链路中，每次 `MAP_PRIVATE` 文件缺页的时候，即使只是读操作，也要经过“磁盘 → Page Cache → 分配私有页 + memcpy 4 KiB → PTE”的完整流程。大量只读文件页（`.rlib`、`.o`、源码）永远不被写，这些分配和复制全部浪费，且多进程映射同一文件时各复制一份，明显加剧了内存压力。

所以优化的核心思路是：对满足条件的只读缺页，直接把 folio 中已就绪的物理页以只读方式装入 PTE，跳过私有页分配和 memcpy。PTE 标记为只读以保证 `MAP_PRIVATE` 语义——进程真正写入时再次缺页（COW），此时才分配私有页、从 folio 复制内容、换为可写 PTE。这样，永远不被写的页零额外成本，被写的页仅在实际写入时付出一次复制代价。新旧链路对比如下：

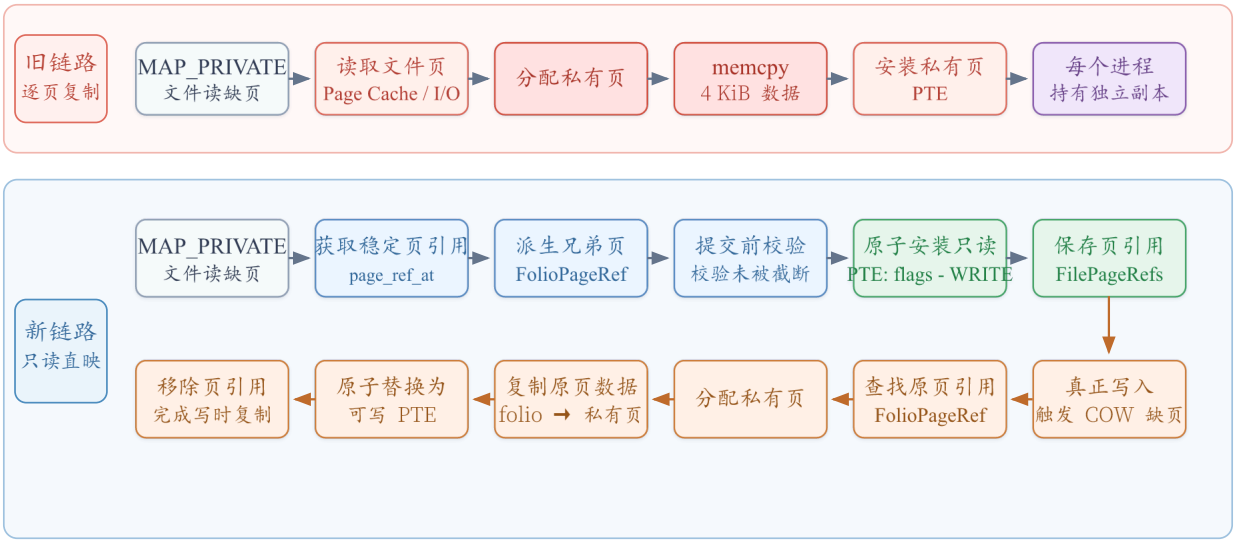


图 10　只读文件页直接映射：旧链路 vs 新链路

具体而言，execute 阶段在锁外调用 `file.page_ref_at(offset)` 获取 `FolioPageRef`——此时 I/O 已完成、folio 处于 uptodate 状态，拿到的是指向页缓存物理页的稳定引用。然后从 folio 派生出同一窗口内的兄弟页，一起交给 commit 阶段批量安装：

```
// CowFaultPlan::execute() —— 只读文件映射快路径（锁外）
if self.size == PageSize::Size4K
    && !self.access_flags.contains(MappingFlags::WRITE) // 非写缺页
    && start == 0 && max_read == self.size as usize      // 完整对齐页
{
    if let Some(fault_page) = file.page_ref_at(fault_file_start)? {
```

```
        // 从 fault folio 派生兄弟页，故障页优先
        self.file_pages.push((self.vaddr, fault_page.clone()));
        for addr in window_start..window_end {
            if addr != self.vaddr {
                if let Some(sibling) = fault_page.folio_page_ref(offset) {
                    self.file_pages.push((addr, sibling));
                }
            }
        }
    }
}
```

commit 阶段重新持读锁后，调用 `file.with_valid_folio_ref` 校验 folio 未被 truncate，然后以 `flags - MappingFlags::WRITE`（强制只读）CAS 安装 PTE，同时将 `FolioPageRef` 存入 `FilePageRefs` 表。后续真正写入触发 COW 时，从该表中查得原来的 `FolioPageRef`，据此分配私有页、复制内容、换为可写 PTE 并释放 folio 引用。

```
// CowBackend::commit_missing_fault() —— 安装只读 PTE
let validated = file.with_valid_folio_ref(&validation_page, || {
    for (addr, page) in pages {
        // CAS 安装 PTE，强制只读（flags - WRITE）
        match pt.map_4k_concurrent(addr, page.paddr(), flags - MappingFlags::WRITE) {
            Ok(()) => {
                file_pages.insert(addr, page);  // 保存 FolioPageRef，PTE 在位期间不释放
            }
            Err(PagingError::AlreadyMapped) => {}  // 并发已安装，跳过
            // ...
        }
    }
});
```

回退到旧慢路径的条件：写缺页、非 4 KiB 对齐、EOF 不完整尾页、页缓存无法返回稳定引用、commit 时 folio 已失效（truncated 或被替换）。

3.5 优化五：ext4 锁外数据读写 + VirtIO 多在途请求

如 2.4 根因所述，一次普通文件读取在 ext4 全局锁内要完成块映射查询并等待磁盘，跨文件 I/O 会被全局串行化。所以本优化分两层来解除串行：extent cache 与锁外数据 I/O（上层），异步 VirtIO 多在途请求（底层）。

3.5.1 extent cache 与锁外数据 I/O

先区分两类工作：extent 映射查询属于短元数据操作，数据 I/O 属于长等待操作。优化思路是把慢操作移出锁外——ext4 锁内只做“查 extent、记映射”，释放锁后再做磁盘 I/O。新旧读路径对比如下：

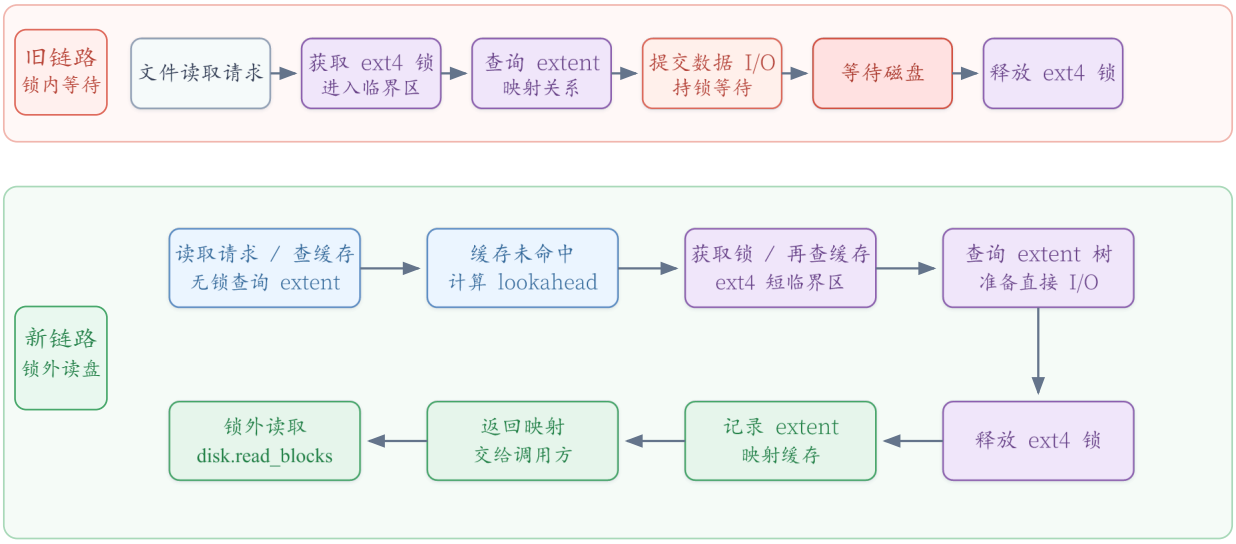


图 11　ext4 锁外数据 I/O：旧链路 vs 新链路

读路径上，extent cache 按 inode 分 64 个 shard，cache miss 时做约 1 MiB 的 lookahead 预取相邻 extent；进入 ext4 锁后必须再次检查 cache——因为排队等锁期间，其他线程可能已经填好了同一映射。稀疏文件的空洞区域没有分配物理块，直接填零返回。

```
fn read_extents(&self, ino, start, count, file_size) -> VfsResult<Vec<BlockExtent>> {
    // ① 先查 extent cache（无锁），命中则直接返回物理块映射
    if let Some(extents) = self.cached_extents(ino, start, count) { return
Ok(extents); }
    // ② 计算 lookahead 窗口（~1 MiB）
    let mapping_count = count.max(lookahead.min(available));
    let extents = {
        let mut fs = self.lock();     // ③ 短暂获取 ext4 锁
        // ④ 锁内再次检查 cache（排队期间可能已被其他线程填好）
        if let Some(extents) = self.cached_extents(ino, start, count) { return
Ok(extents); }
        let extents = fs.map_blocks(ino, start, mapping_count)?;  // 查 extent 树
        fs.prepare_direct_data_access(&extents)?;  // 准备直接 I/O
        extents
    };  // ⑤ 释放 ext4 锁
    // ⑥ 锁外：缓存 extent 映射，返回给调用方去做 disk.read_blocks
    self.cache_extents(ino, &extents);
    self.cached_extents(ino, start, count).ok_or(VfsError::Io)
}
```

写路径更复杂，因为写入可能涉及分配新块和更新文件长度，不能完全绕开 ext4：锁内分配或确认块映射并准备缓存一致性，锁外写数据块，最后再短暂进入 ext4 提交文件长度与元数据更新。文件中间非对齐部分块写仍走 lwext4 的 read-modify-write 安全路径。

```
fn write_mapped(&self, ino, buf, offset, file_len) -> VfsResult<usize> {
    self.invalidate_extent_cache(ino);  // 写操作先失效缓存，下次读重新查映射
    let (block_size, extents) = {
        let mut fs = self.lock();        // 获取 ext4 锁
        // 非对齐部分块写（修改块中间一小段）→ 回退 lwext4 安全路径
        if !offset.is_multiple_of(block_size)
```

```
            || (!buf.len().is_multiple_of(block_size) && write_end < file_len)
        { return fs.write_at(ino, buf, offset); }
        // 对齐整块写：锁内分配/确认块映射
        let extents = fs.allocate_blocks(ino, logical_block, block_count)?;
        (block_size, extents)
    };  // 释放 ext4 锁
    // 锁外写数据块 → disk.write_blocks(...)
    self.lock().commit_file_size(ino, new_size)?;  // 最后短暂入锁提交长度
}
```

缓存一致性方面，任何可能改变块分配的操作必须失效对应 inode 的 extent cache，否则旧映射会指向已被回收或重新分配的物理块。

```
/// 写操作、truncate、删除/重建 inode 前调用，使旧映射失效
pub(crate) fn invalidate_extent_cache(&self, ino: u32) {
    self.extent_cache_for(ino).write().remove(&ino);
}
```

此外有两项配合性改进。一是将 ext4 全局锁从自旋锁改为阻塞 Mutex：锁外 I/O 已经把持锁时间从“等磁盘的毫秒级”缩短为“查 extent 的微秒级”，此时再用自旋锁空转等锁已无必要，改为阻塞 Mutex 后等锁线程直接睡眠，把 CPU 让给其他编译任务。

二是脏页积累后批量写回：`CachedFile` 先将多个 folio 的脏页积累在内存中，汇总后再一次进入 ext4 提交文件长度和元数据更新，而不是每写回一个 folio 就获取一次 ext4 锁。这样减少了 ext4 锁的获取频率，避免数据 I/O 已经移出锁外、元数据提交又成为新的串行点。

3.5.2 底层块设备：异步 VirtIO 多在途请求

ext4 锁外 I/O 解决了上层串行化，但如果块设备层仍然一次只允许一个请求在途，不同文件的 I/O 还是被设备互斥锁串在一起。因此底层加入异步 VirtIO 块设备层，让多个 I/O 请求可以同时在 VirtIO 队列中等待设备处理。新旧链路对比如下：

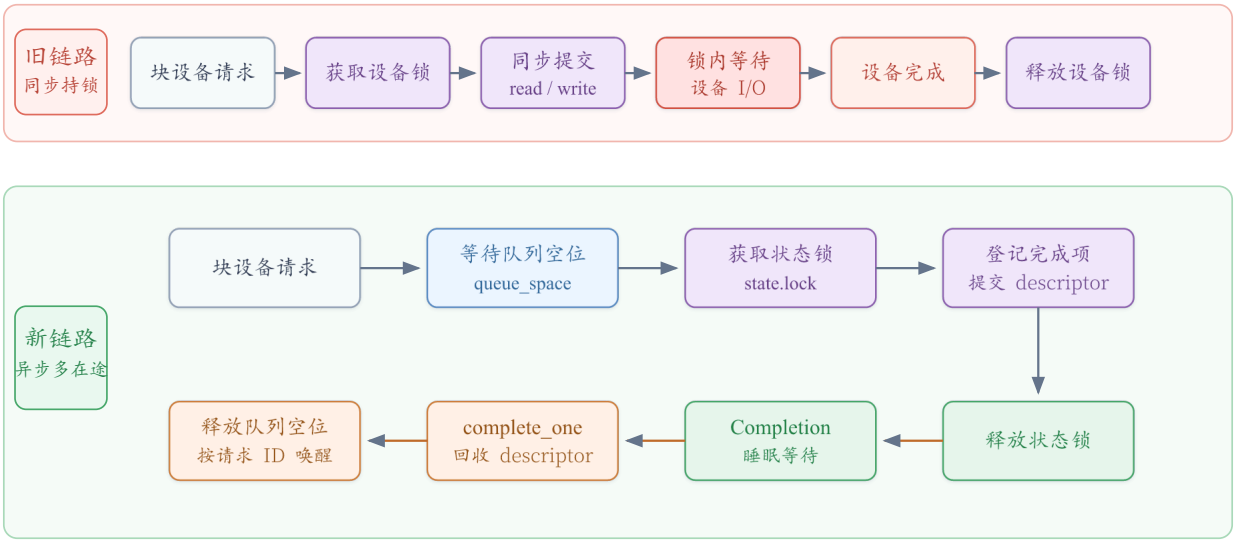


图 12　异步 VirtIO 块设备层：旧同步链路 vs 新异步链路

旧链路中，提交线程持设备锁完成”提交 → 等 I/O → 释放锁”的完整流程，一次只有一个请求在设备上。新链路中，设备锁只保护 descriptor 的提交与回收（微秒级），提交线程把请求放入 VirtIO 队列、通知设备后立即释放锁，然后在自己独立的 `Completion` 上睡眠等待。I/O 完成后由 completion

task 回收 descriptor 并按请求 ID 唤醒对应的提交线程。设备锁不覆盖磁盘完成时间，最多 16 个请求可同时在途。

核心数据结构为每请求一个 `Completion`，内含结果槽和等待队列：

```
struct Completion {
    result: Mutex<Option<DevResult>>,  // I/O 完成后由 completion task 填入
    wait: WaitQueue,                   // 提交线程在此睡眠等待
}

impl Completion {
    fn finish(&self, result: DevResult) {       // completion task 侧调用
        *self.result.lock() = Some(result);
        self.wait.notify_one(false);            // 唤醒提交线程
    }
    fn wait(&self) -> DevResult {               // 提交线程侧调用
        self.wait.wait_until(|| self.result.lock().is_some());
        self.result.lock().take().unwrap()
    }
}
```

提交路径：持设备状态锁 → 分配请求 ID → 调用 `submit_read/submit_write` 放入 VirtIO 队列 → 通知设备 → 释放状态锁 → `completion.wait()` 睡眠。队列满时在 `queue_space` 上等待，每完成一个请求唤醒一个提交者避免惊群。

```
unsafe fn submit(&self, write: bool, block_id, buffer) -> DevResult {
    // 系统启动早期中断未就绪 → 仍走同步 I/O
    if !ASYNC_BLOCK_IO_READY.load(Ordering::Acquire) {
        return self.submit_sync(write, block_id, buffer);
    }
    self.0.queue_space.wait_until(|| self.queue_has_space());  // 等队列有空位
    let id = self.0.next_request.fetch_add(1, Relaxed);
    let completion = Arc::new(Completion::new());
    {
        let mut state = self.0.state.lock();                    // 短暂持设备锁
        state.pending.insert(id, completion.clone());
        state.device.submit_read(id, block_id, buffer)?;      // 提交 descriptor
    }                                                             // 释放设备锁
    self.0.work.notify_one(false);              // 通知 completion task
    return completion.wait();                   // 睡眠等待 I/O 完成
}
```

`flush` 会先等待全部在途请求排空后再独占执行，可以保证文件系统一致性。

3.6 优化六：地址空间回收与 TLB 刷新

前五项核心优化把文件缺页与内存管理主链路缩短后，进程退出和 `exec` 的地址空间清理与 TLB 同步成为新的可见热点。BuildStorm 上百个 rustc/cc1 进程频繁 fork/exec/exit，每次退出都要销毁一个包含大量映射的地址空间——旧实现中这个销毁过程既慢又阻塞前台线程，且每释放一页就触发一次全核 TLB shootdown。

旧链路：退出线程持有 `AddrSpace` 写锁 → 逐 area → 逐 4 KiB 页遍历整个虚拟地址范围（不论是否实际映射过）→ 每页 `pt.unmap` + `pt.finish`（每页一次跨核 TLB 刷新）→ 逐页查 frame refcount

表并释放 → 最后释放锁。旧链路中多次“逐页”是核心问题——既逐页扫、也逐页刷 TLB、还要逐页释放。总览新旧链路对比如下：

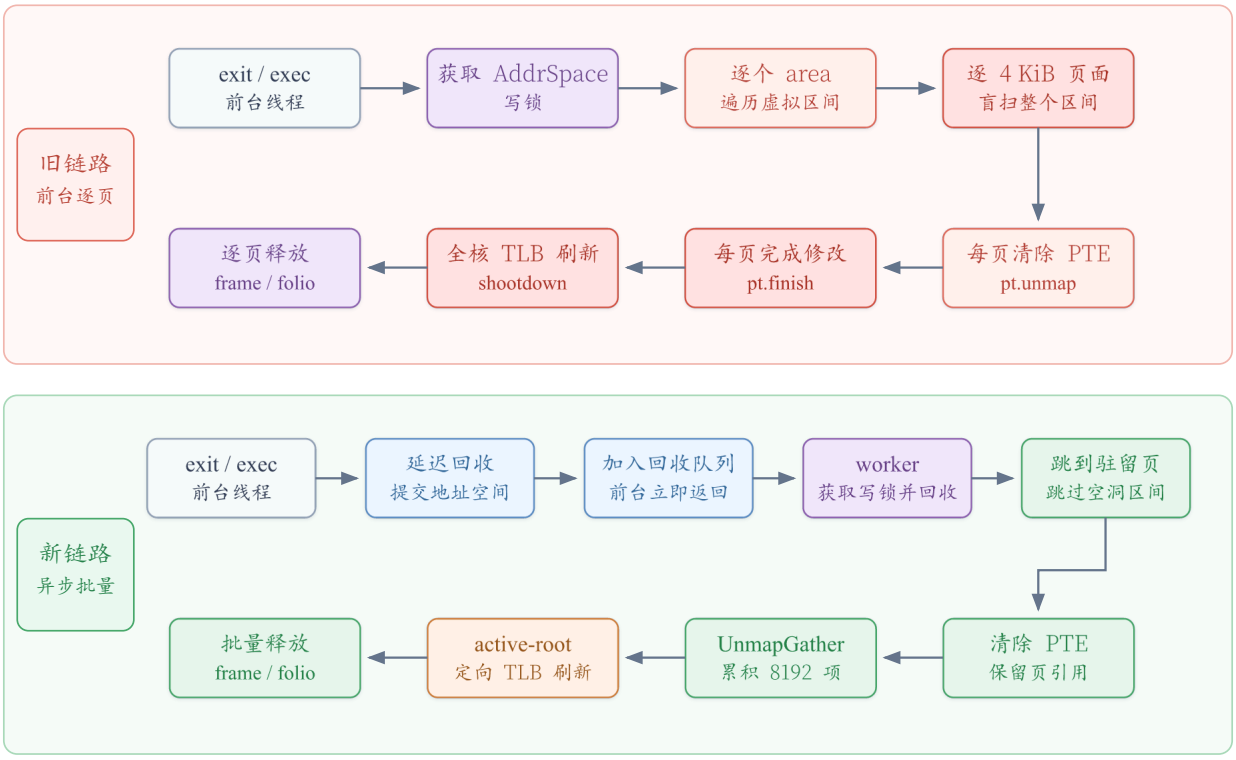


图 13　地址空间回收：旧同步逐页 vs 新异步批量 + 定向 TLB

优化的核心思路具体是四个层面的改进。

其一，异步回收。退出/exec 的前台线程通过 `defer_address_space_reclaim` 把已死亡的 `AddrSpace` 放入回收队列后立即返回，不再阻塞前台。后台有多个回收 worker 独立从队列中取出地址空间并执行 `clear()`，退出线程不再持有地址空间写锁等待清理完成。

```
pub(crate) fn defer_address_space_reclaim(aspace: Arc<AddrSpaceLock>, ...) {
    RECLAIM_QUEUE.lock().push_back(AddrSpaceReclaim { aspace, owner });
    start_reclaim_worker();  // 按需启动 worker，最多 cpu_num 个
}
```

其二，仅扫驻留页。旧实现用 `pages_in(range)` 逐 4 KiB 推进，遍历整个 VMA 范围的全部虚拟地址。新实现中 `unmap_for_clear` 使用页表自身的 `next_present_mapping(cursor, end)` 接口，沿页表树直接跳到下一个存在 PTE 的地址，整段空的中间页表层级被一次性跳过，不再逐页探测空洞。

其三，UnmapGather 合并 TLB 刷新。旧实现中每个 PTE 的 `pt.unmap` 后紧跟 `pt.finish()`，一个进程退出触发成千上万次独立的跨核 TLB 刷新。新实现中，`UnmapGather` 暂存被清除的 frame/folio 引用（匿名页的物理地址或文件页的 `FolioPageRef`），累计最多 8192 个驻留映射后才执行一次统一的 TLB shootdown，完成后一次性释放这批所有权。顺序是先清 PTE → 保留引用 → 完成远端 TLB 失效 → 再释放物理页，不可颠倒。

```
pub(crate) fn clear(&mut self) {
    let mut gather = UnmapGather::new();
    for area in areas.iter() {
        // unmap_for_clear: 沿 next_present_mapping 前进，每页清 PTE，
        // 将 frame/folio 引用 push 进 gather，满 8192 就先 shootdown 再释放
        area.backend().unmap_for_clear(area.va_range(), &mut pt, &mut gather)?;
```

```
    }
    gather.finish(&mut pt);  // 最后一轮 shootdown，释放剩余所有权
}

struct UnmapGather { pages: Vec<DeferredUnmapPage>; }  // 最多 8192 项

impl UnmapGather {
    fn finish(&mut self, pt: &mut PageTableMut) {
        pt.finish();  // 一次 TLB shootdown，覆盖本轮所有清过的 PTE
        for page in self.pages.drain(..) {
            match page {
                DeferredUnmapPage::Anonymous { paddr, page_size } =>
drop_referenced_frame(paddr, page_size),
                DeferredUnmapPage::File(page) => drop(page),  // 释放 FolioPageRef
            }
        }
    }
}
```

其四，定向 TLB 刷新（active-root）。这是基于前面合并刷新 TLB 基础上的进一步优化。旧实现中任何页表修改都通过 `pt.finish()` 无条件向全部 CPU 发送 TLB 刷新，即使大部分 CPU 根本没运行过这个地址空间。新实现中每个 CPU 在切换用户页表根之前，通过原子数组 `ACTIVE_PAGE_TABLE_ROOTS` 发布当前页表根的物理地址。TLB shootdown 时根据 root 筛选：RISC-V 只向仍运行该地址空间的 hart 发 SBI `remote_sfence_vma`，LoongArch 只向选中 CPU 发 IPI 执行按 VA 的 `invtlb`。root 为 0 或无法确定时保守刷新所有 CPU。CPU 切入新 root 时的顺序是先发布 root、再安装页表、随后本地全刷，避免与并发 shootdown 竞态时漏掉目标 CPU。

```
// 每个 CPU 发布当前用户页表根
static ACTIVE_PAGE_TABLE_ROOTS: [AtomicUsize; CPU_NUM] = ...;

// RISC-V: 仅向运行目标地址空间的 hart 发送远程 TLB 刷新
let mask = cpu_bits.iter().enumerate().filter(|(cpu, _)| {
    ACTIVE_PAGE_TABLE_ROOTS[cpu].load(Acquire) == root
});
sbi_rt::remote_sfence_vma(HartMask::from_mask_base(mask, base), start, size);
```

3.7 其他优化

核心优化完成后，缺页提交路径上的引用计数、PTE 安装与 VFS 名字解析/元数据读取成为剩余热点。以下优化属于细碎改进：frame 引用计数直查、PTE/COW/Folio 快速提交，以及 VFS 目录项与 inode 元数据并发缓存。

3.7.1 缺页提交微路径

核心缺页链路缩短后，引用计数查询、PTE 提交和 COW 所有权记录开始出现在火焰图中。以下四项微优化各针对一条局部热点，按"旧做法 → 问题 → 新做法"分述。

frame 引用计数直查。旧实现将每张物理页的引用计数存入 `BTreeMap<PhysAddr, Arc<SpinNoIrq<FrameRefCnt>>>`，分配、COW、释放时都需要 BTree 查找且全程关中断。新实现根据物理内存的起始地址和页号计算紧凑数组下标——`(paddr - base) / 4KiB` 直接定位到 64 路分片的槽位，O(1) 查表。同时将临界区从"关中断"缩短为"仅禁止抢占"，因为普通引用增减不涉及中断上下文，只有 PTE 安装/移除的原子 CAS 侧才需要更重的同步。

```
// 新：紧凑数组，物理地址 → 直接计算下标
struct FrameRefTable {
    shards: [SpinPreempt<BTreeMap<usize, FrameRefState>>; 64],
}
fn location(paddr: PhysAddr) -> FrameRefLocation {
    let offset = paddr.as_usize() - base.as_usize();
    FrameRefLocation { shard: (offset / 4096) % 64, index: offset / 4096 }
}
```

PTE commit 删除重复查询。旧实现中 missing-fault commit 先调用 `pt.query` 完整走一次多级页表遍历确认槽位为空，再执行 `pt.map` 做第二次页表遍历并 CAS 安装。新实现直接调用 `pt.map_4k_concurrent`，CAS 返回的旧 PTE 值即可区分三种情况：空槽（本线程安装成功）→ `Handled`；已存在等价映射 → `Handled`（竞争已处理）；状态不满足 → `Stale` 走有限重试。删掉一次完整页表遍历。

COW 文件页内联槽。`MAP_PRIVATE` 文件映射只读直接安装 folio 页时，必须记录"这个 PTE 正在固定哪个 `FolioPageRef`"，以便后续 COW 写缺页时查得原引用并替换。旧实现无论多少文件页一律进 BTree。新实现优先使用 backend 内的一个内联槽（仅一个条目），超出容量才升为 BTree overflow；删除内联项后从 overflow 提升一项填补。纯匿名映射根本不创建文件页所有权表。

folio fault-around。从故障页所在的 folio 派生同窗口内的兄弟页，一次 `with_valid_folio_ref` 校验整个 folio 的有效性后，循环内按地址由低到高逐一 CAS 安装多张 4 KiB PTE，最多覆盖一个 folio（16 页）。故障页优先安装保证原始缺页先被满足，兄弟页冲突可跳过不影响整体。相对当时的正确性基线，BuildStorm 单轮改善约 7.44%。

3.7.2 VFS 目录项与 inode 元数据并发缓存

BuildStorm 不仅密集读取文件内容，还频繁执行 `lookup`、`stat`、创建临时文件和重命名。ext4 数据 I/O 已经移出全局锁，但如果 VFS 层在做一次冷目录查询时持有目录缓存写锁等待磁盘，其他线程的缓存命中仍会被连带阻塞。

目录缓存方面，旧实现只缓存 positive dentry（已确认存在的名字），对不存在的名字每次都要进入 ext4 重复查询。新实现同时缓存 negative dentry，把不存在的名字也纳入缓存，构建系统反复探测同一不存在的路径时直接返回 `NotFound`。每个目录设置 16 个按文件名哈希的睡眠锁：同名并发 miss 只允许一个线程实际执行文件系统 lookup，其他等待者醒来后重新检查缓存；不同名字仍可并行。lookup 期间不持有目录 cache 写锁，查询完成后通过 `cache_generation` 检查期间是否发生 namespace 修改，只有代数一致才发布结果。create/unlink/link/rename 等修改操作由目录 `mutation_lock` 串行提交，跨目录 rename 按地址顺序取两把锁避免死锁。

inode 元数据缓存方面，旧实现用一把全局缓存锁保护所有 inode 的属性读取。新实现改为 64 路分片，同一 inode 的加载与回写使用睡眠锁合并（避免重复读取），不同 inode 的 `stat`/属性读取不再争同一把锁。

第 4 章 实验分析

4.1 实验环境与方法

本章数据在同一台 Ubuntu x86_64 主机上采集。宿主机为 Intel Core i9-14900HX（32 个逻辑 CPU）；被测环境统一为 LoongArch64 `virt` 平台、QEMU 10.0.2、8 vCPU、8 GiB 内存，磁盘使用 `sdcard-la-pub.img.gz` 解压得到的 `sdcard-la.img`，并通过 `-snapshot` 保证每轮从相同状态启动。宿主机与正式测评机硬件不同，因此实验数据会与测评机有所差异，但阶段间横向比较使用完全相同的 QEMU、镜像和参数。

Linux 基线与 Moss 十个阶段均串行执行三次，避免并发实验互相抢占宿主机资源；报告采用赛题输出 `BUILDSTORM_COMPILE` 中有效编译时长的中位数。每轮同时以 1 秒间隔采集 QEMU 进程 CPU、RSS 和进程 I/O 计数；异常退出、panic 或长时间无进展的运行不进入正式样本并单独留档。正式测量内核关闭 profiler；CPU 火焰图以及 `stage-00/03/04/09` 的 mutex、`mprotect` 等待图均来自独立插桩运行，完全不计入耗时统计。

4.2 性能数据展示

为方便展示，以下各表中阶段编号与优化措施的对应关系为：

```
00 为正确性基线；
01 为 folio 与只读文件页直接映射；
02 为多核 futex、管道与退出路径稳定化；
03 为 per-CPU 物理页缓存（PCP）；
04 为 mprotect 与批量 TLB 刷新；
05 为 negative dentry 与 inode 分片；
06 为异步 VirtIO、extent cache 与锁外 I/O；
07 为 fault-around、COW 微路径与 active-root 修复；
08 为文件系统、COW、页分配与锁竞争综合优化；
09 为 main 分支最终稳定版本。
```

4.2.1 编译耗时

优化前基线中位数为 993.08 秒，最终版本降至 354.27 秒，累计下降 64.33%，约 2.80 倍加速。相同条件下 Linux 基线中位数为 413.31 秒，最终 Moss 比本机 Linux 基线快 14.28%。阶段 02 和阶段 06 出现小幅回退，说明单项提交效果会受到阶段组合与宿主机噪声影响；三次测量的范围和原始数据均完整保留。

阶段	累积优化	耗时 (s)	相邻变化	相对基线
Linux	Ubuntu 基线	413.31	—	—
00	正确性基线	993.08	—	—
01	folio + 只读直映	424.94	-57.21%	-57.21%
02	多核稳定化	443.20	+4.30%	-55.37%
03	PCP	426.87	-3.68%	-57.02%
04	mprotect + TLB	398.19	-6.72%	-59.90%
05	VFS 并发缓存	397.44	-0.19%	-59.98%
06	异步 I/O	398.56	+0.28%	-59.87%
07	fault/COW/TLB	389.50	-2.27%	-60.78%
08	综合锁竞争优化	372.30	-4.42%	-62.51%
09	最终版本	354.27	-4.84%	-64.33%

表 4-1　各阶段 BuildStorm 编译耗时

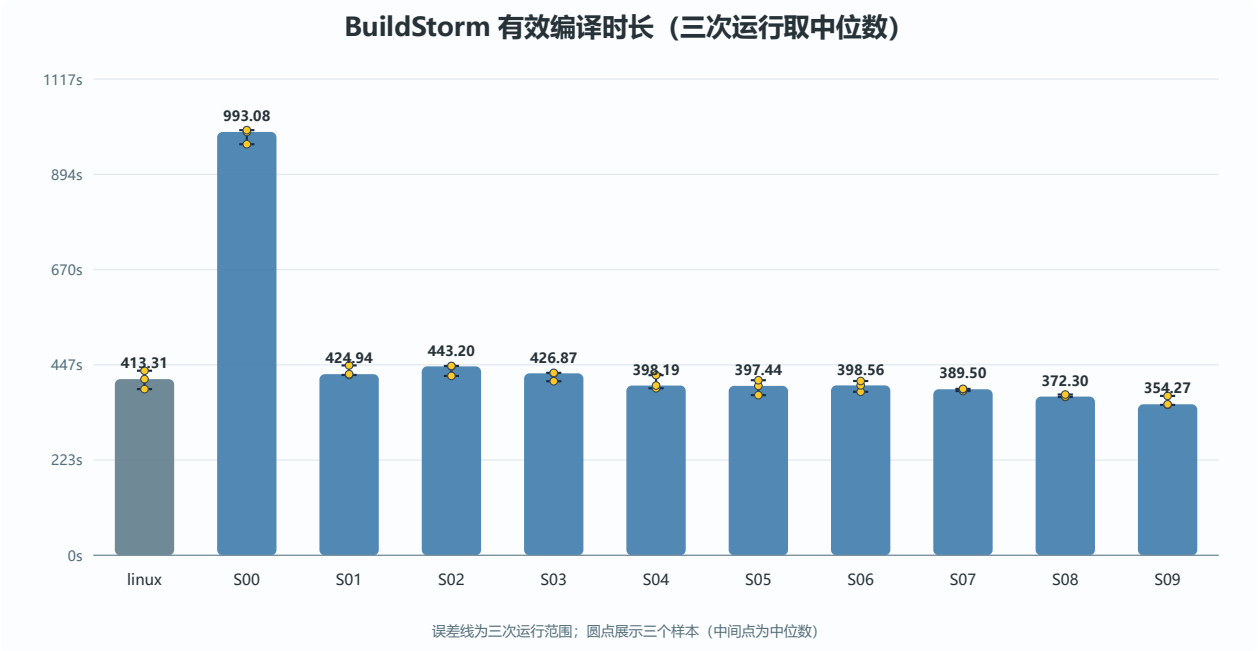


图 14　三次串行测量的 BuildStorm 有效编译时长；柱为中位数，误差线为最小值至最大值

4.2.2　CPU 并行度

把 QEMU 进程每秒 `%CPU / 100` 记为“有效核数”。Moss 基线平均使用 3.077 核，最终版本为 3.769 核；6～8 核区间占比从 0.20% 提高到 35.98%。Linux 基线的平均值与 6～8 核占比分别为 4.432 核和 46.79%。最终版本的平均核数并非所有阶段最高，但它以更少的累计 CPU 时间完成了工作，因此应同时结合墙钟时间和核数分布判断。

阶段	平均有效核数	中位数	**6～8** 核占比
Linux	4.432	4.923	46.79%
00	3.077	2.923	0.20%
01	3.971	2.442	37.97%
02	3.954	2.543	37.92%
03	3.957	2.444	37.01%
04	3.798	2.082	34.60%
05	3.845	2.115	35.26%
06	3.849	2.243	36.68%
07	3.808	2.124	36.76%
08	3.678	1.855	34.22%
09	3.769	2.034	35.98%

表 4-2　各阶段 CPU 并行度

表 4-2 中最明显的变化不是平均核数增加了多少，而是优化后的内核终于能够进入 6～8 核的高并行状态。为了看清这种变化在整个编译过程中的占比，将每秒采样到的有效核数按区间统计如下。

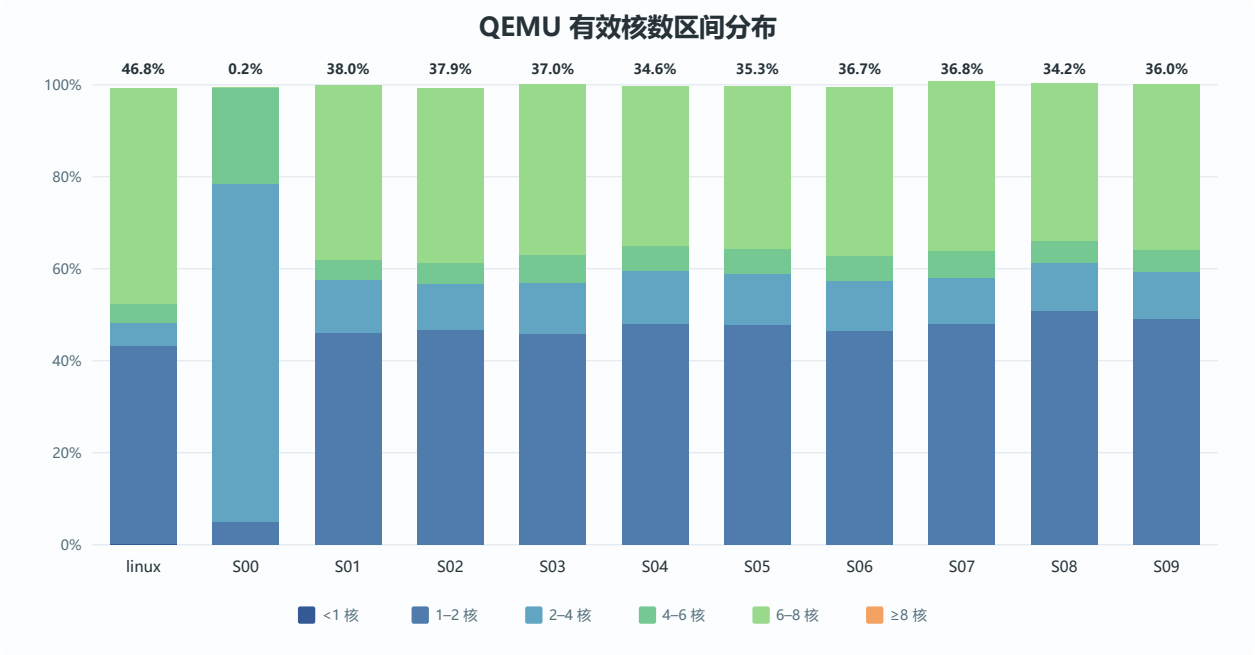


图 15　各阶段有效核数区间占比

优化前，73.62% 的时间只能使用 2～4 核，6～8 核占比仅为 0.20%，说明多个编译任务虽然已经启动，却大量阻塞在缺页、页缓存和文件系统路径上。从 `stage-01` 开始，6～8 核占比立即提升到 37.97%，后续各阶段也基本保持在 34%～38%。这与 folio、只读文件页直接映射以及后续锁粒度优化的目标一致：慢速 I/O 和页缓存操作不再长时间串行阻塞其他任务，多个 rustc 进程因而能够真正并行推进。最终版本的高并行占比为 35.98%，仍低于 Linux 的 46.79%，说明锁竞争得到显著缓解，但尚未完全消除。

这种改善在时间轴上更加直观。下面选取首末版本中最接近各自三次中位数的一轮，对逐秒数据作 5 秒移动平均，观察高并行阶段何时出现、持续多久，以及整个编译过程何时结束。

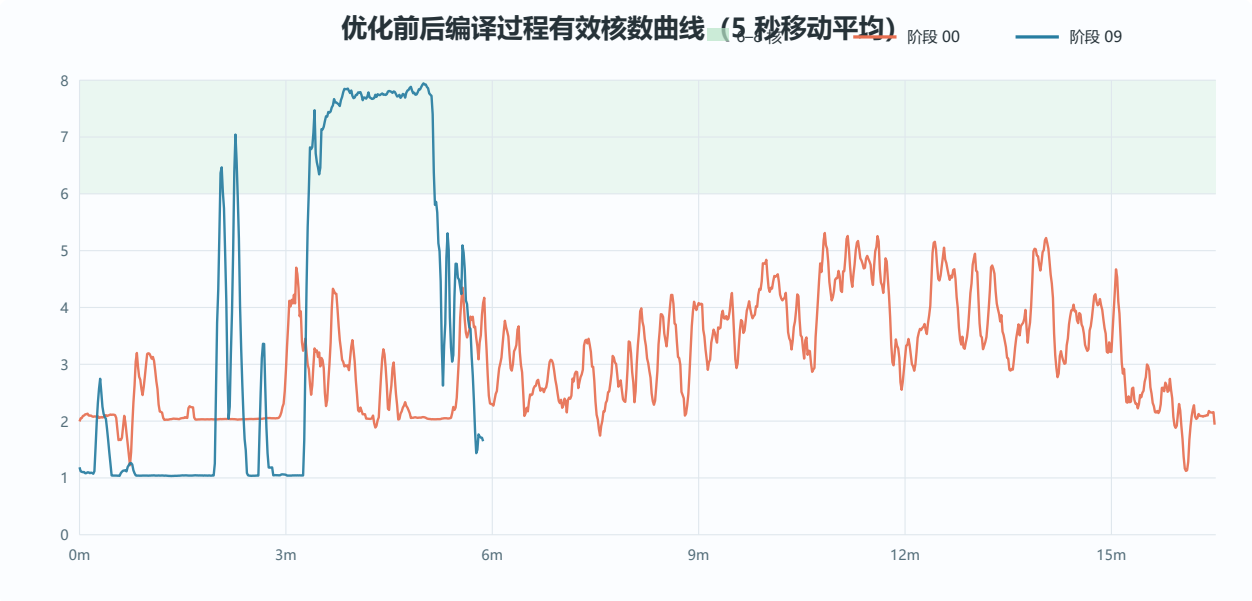


图 16　首末阶段单轮中位数样本的有效核数时间曲线；浅色区域为 6～8 核区间

基线运行约 16 分钟，大部分时间只有 2～5 核在工作，始终没有形成持续的满核区间。最终版本在约第 3～5 分钟保持接近 8 核，并在约 6 分钟时完成编译。也就是说，优化不仅让更多任务可以同时运行，还缩短了每个任务在内核路径中等待和重复处理的时间。正因如此，最终平均有效核数只由 3.077 增加到 3.769，编译时间却由 993.08 秒降到 354.27 秒：这 2.80 倍加速同时来自并行度提升和内核额外工作的减少。

4.2.3　内存与 I/O

提升页缓存与 I/O 并行度会改变内存和磁盘访问方式，因此还要确认加速是否以过高的资源开销为代价。RSS（Resident Set Size，常驻内存集）表示 QEMU 进程实际驻留在宿主机物理内存中的大小，其中既包含客体已经触及的内存，也包含 QEMU 自身开销；它适合比较各阶段的整体内存压力，但不能直接等同于 Moss 内核自身的内存用量。这里的 I/O 同样来自宿主机对 QEMU 进程的计数，只用于同一环境下的阶段间比较，不能视为客体文件系统的精确磁盘流量。

阶段	平均 **RSS (MiB)**	最大 **RSS (MiB)**	估算读取 **(GiB)**	估算写入 **(GiB)**
Linux	2331.6	2751.1	0.010	0.306
00	1864.1	2148.1	0.002	0.397
01	2138.3	2499.4	0.004	0.303
02	2132.5	2543.7	0.000	0.304
03	2259.3	2851.7	0.000	0.301
04	2247.0	2778.4	0.001	0.305
05	2286.8	2884.4	0.002	0.299
06	2330.5	2925.1	0.000	0.300
07	2330.2	2866.6	0.005	0.298
08	2336.0	2846.7	0.000	0.079
09	2337.7	2888.2	0.000	0.079

表 4-3　各阶段内存与宿主机 I/O 估算（正式窗口）

从表 4-3 可以看出，性能提升确实伴随一定的常驻内存增长，但增长在早期优化后很快趋于稳定；与之相比，后期版本的宿主侧写入量出现了更明显的下降。各阶段的变化如下图所示。

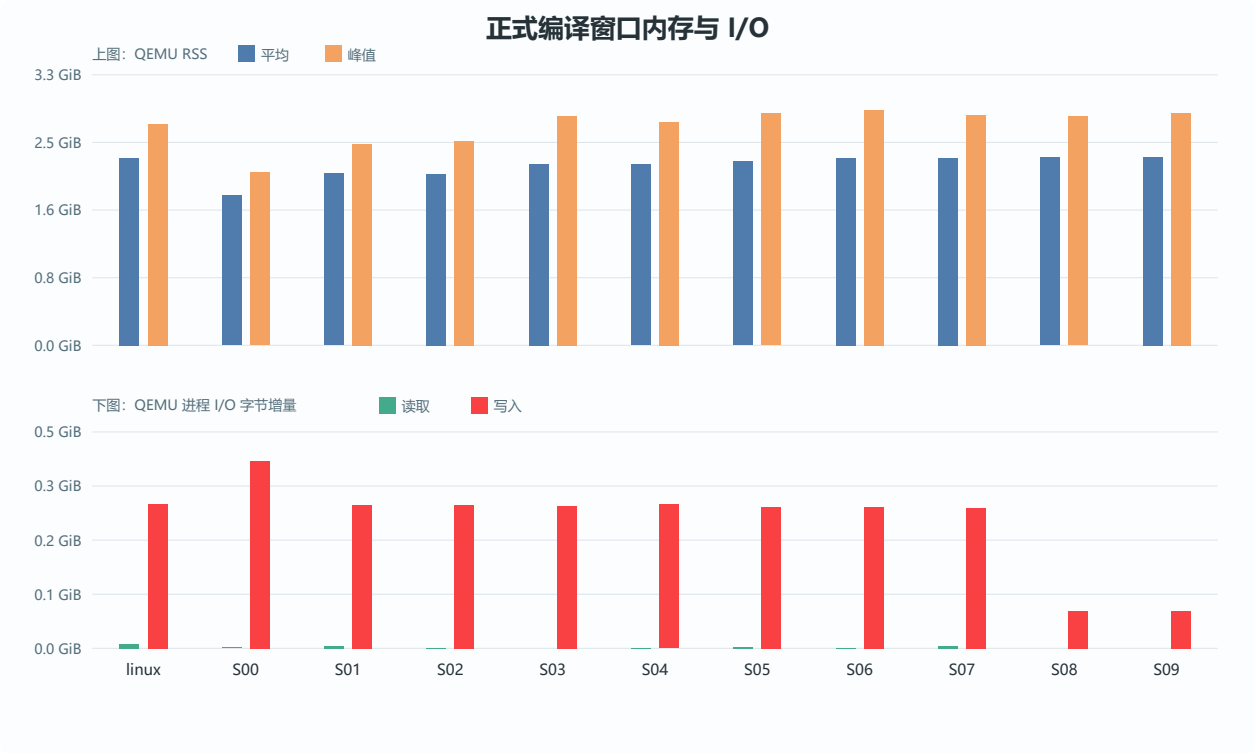


图 17　各阶段内存占用与宿主机 I/O 估算（上：平均与峰值 RSS；下：估算读取与写入）

`stage-01` 合入 folio 与只读文件页直接映射后，平均 RSS 由 1864 MiB 增至 2138 MiB，说明更积极的页缓存和批量处理确实需要保留更多常驻数据。此后即使继续加入 PCP、VFS 缓存和异步 I/O，平均 RSS 也只是缓慢上升并稳定在约 2.2～2.3 GiB，最终版本为 2338 MiB，没有随性能继续提升而持续膨胀。因此，这部分内存增长可以视为用有限缓存空间换取更少缺页、重复读取和锁内等待。

宿主侧写入量从基线的 0.397 GiB 降到 `stage-01` 的约 0.303 GiB，`stage-08` 后又进一步降至约 0.079 GiB，最终较基线减少约 80.02%。这说明后期文件系统、COW 和页分配路径的综合优化不仅缩短了执行时间，也减少了宿主侧实际观察到的写入工作。读取量受宿主页缓存和 `-snapshot` 机制影响较大，不宜单独解读。总体来看，64.33% 的耗时下降只带来了有限且稳定的常驻内存增长，同时明显降低了写入压力。

4.3　LoongArch 火焰图对比

首末阶段各单独运行一次带 QEMU gdbstub 的采样实验，每 2 秒暂停一次并采集 8 个 vCPU 的调用栈。该方法会扰动运行，只用于定位热点，不参与性能结果。为补足 CPU 图看不到的阻塞时间，又对 `stage-00/03/04/09` 各做一次独立的 LoongArch 等待插桩运行。探针只在 BuildStorm 正式窗口开启，mutex 记录实际进入睡眠的等待区间，`mprotect` 每 32 次采样一次并折算。四轮均为 `ok=true`、8 核、LoongArch，且 `dropped_table` 与 `dropped_pending` 都为 0。

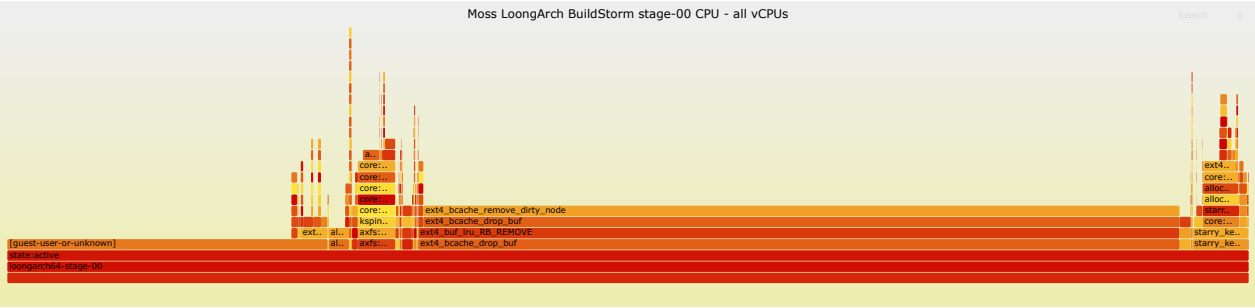


图 18　优化前 LoongArch CPU 火焰图（4112 个 vCPU 样本）

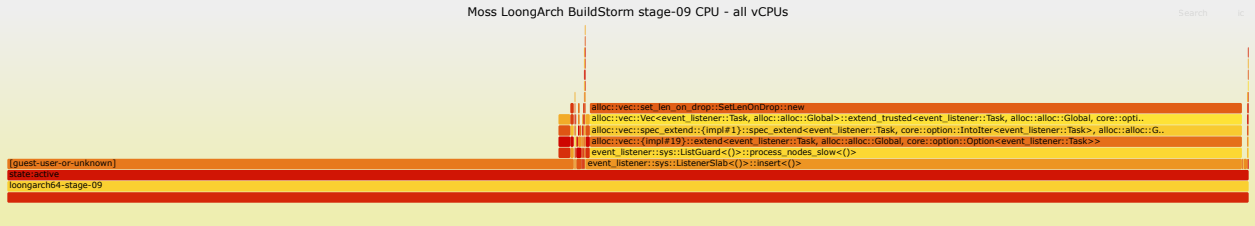


图 19　最终版本 LoongArch CPU 火焰图（1464 个 vCPU 样本）

基线共取得 4112 个 vCPU 样本，其中 3171 个解析为内核符号；`ext4_bcache_drop_buf` 的包含占比约为 80.48%，`ext4_bcache_remove_dirty_node` 的包含占比约为 79.00%，后者作为叶节点约占全部样本的 60.92%，形成非常集中的 ext4 buffer-cache 热塔。最终版本取得 1464 个样本，其中 814 个解析为内核符号，基线中的 ext4 热塔消失，热点迁移到更分散的调度、事件与分配路径。两轮样本数因总运行时间不同而不同，因此这里只比较热点形态，不以火焰图宽度计算加速比。

等待图给出了更直接的前后验证。优化前的基线中，mutex 睡眠等待累计达到 2401.96 秒，其中 COW 文件缺页路径占 84.92%；图中几乎贯穿全部宽度的 `CowBackend::alloc_new_at` 热塔表明，多核编译任务长期阻塞在同一条缺页大锁链上。

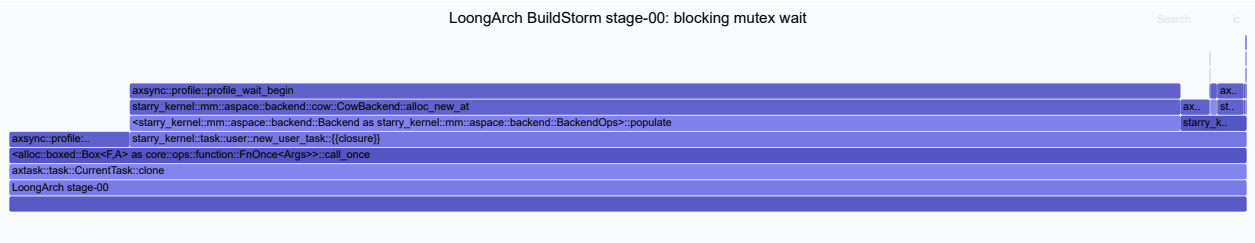


图 20　优化前 stage-00 LoongArch mutex 等待火焰图

完成只读直映、folio 和 PCP 等前期优化后，`stage-03` 的 mutex 累计等待降至 3.15 秒，相比基线减少 99.87%。下图中原先占据主体的 COW 文件缺页热塔已经消失，剩余等待分散在 ext4、页缓存、管道和任务管理等路径。这说明优化不仅缩短了编译时间，也实际拆除了造成多核串行化的主要等待链。

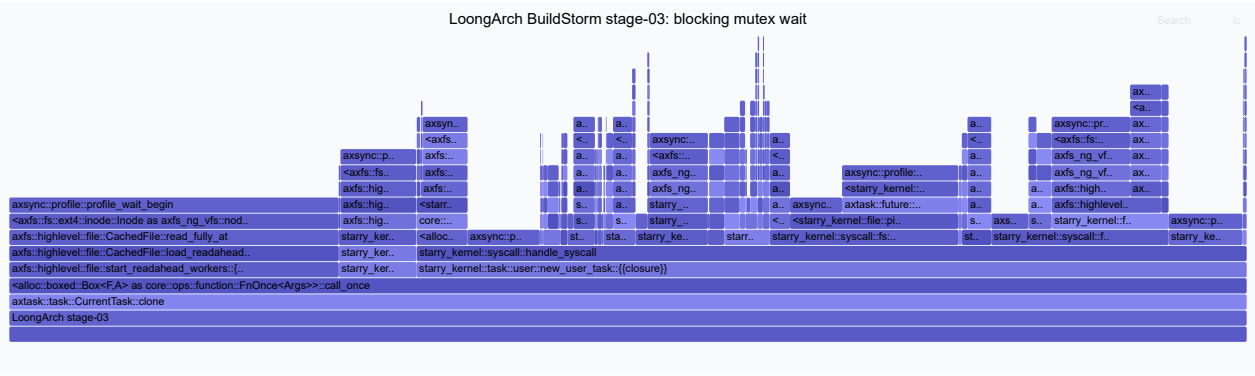


图 21　前期核心优化后 stage-03 LoongArch mutex 等待火焰图

最终版本的累计 mutex 等待为 3.81 秒，虽然受剩余异步任务和运行波动影响略高于 `stage-03`，但依然没有重新出现单一缺页大锁主导的形态。`mprotect` 的折算累计延迟则在 `stage-03` → `stage-04` 从 1.907 秒降至 0.468 秒（-75.45%），与第 3 章 per-VMA 和批量 TLB 设计的目标一致。

4.4　小结

三次串行测量的中位数显示，BuildStorm 从 993.08 秒降至 354.27 秒（-64.33%），6～8 核占比从 0.20% 升至 35.98%，累计 CPU 时间从约 3069 CPU·s 降至 1335 CPU·s；火焰图中原先集中的 ext4

buffer-cache 热点同时消失，COW 文件缺页造成的累计 mutex 等待也下降 99.87%。耗时、并行度与热点迁移三组证据共同支持第 2 章的根因判断和第 3 章的优化方向。

第 5 章 AI 使用说明与可复现步骤

5.1 AI 使用说明

本项目由团队成员确定性能优化目标、技术路线和实验方案，并负责最终的结果复核与方案验收。在此基础上，开发过程中使用大型语言模型（gpt-5.6-sol）协助完成了以下工作：

- 代码实现：AI 根据团队确定的优化方案完成代码撰写，团队成员对并发安全、锁顺序、内存一致性和 TLB 刷新等关键逻辑进行审查，并决定代码的修改与合入；
- 实验监测：AI 持续监测 BuildStorm 实验过程，识别卡死、panic、异常退出和无效样本，并按照既定策略执行重试；实验记录及最终数据由人工复核；
- 资料与版本分析：AI 辅助搜集相关技术资料，对比不同提交和版本的代码变化，整理各阶段的优化内容及其依赖关系；
- 文档整理：AI 对文字材料进行整理和润色，并结合当前代码、提交记录与实验数据，排查文档中的事实偏差、过时数值和表述问题。

AI 主要承担代码撰写、实验执行与资料整理等工作，优化目标、方案选择、结果判断以及最终提交仍由团队成员负责。文档中的性能数据全部来自实际运行结果，不使用 AI 推测的数据。

5.2 可复现步骤

5.2.1 使用仓库脚本运行 BuildStorm

仓库已提供 LoongArch 和 RISC-V 本地运行脚本。运行前在 `src/init_final.sh` 中关闭 cagent、启用 BuildStorm：

```
# src/init_final.sh
cagent=0
buildstorm=1
```

随后在仓库根目录选择需要测试的架构：

```
# LoongArch64
./scripts/run_la.sh

# RISC-V64
./scripts/run_rv.sh
```

脚本会先编译当前工作区，再以 `-snapshot` 启动相应镜像；串口输出分别保存在 `docs/output_la.txt` 和 `docs/output_rv.txt`。出现 `BUILDSTORM_COMPILE ... ok=true` 以及 BuildStorm 组结束标记，表示测试成功。这一入口用于日常正确性验证；第 4 章数据应使用后面的固定实验流程。

5.2.2 复现版本演进

开发过程遵循“采集热点—检查代码—实现优化—运行 BuildStorm—比较结果”的闭环。`docs/buildstorm-report/experiments/la-20260813/stages.tsv` 保存了可完整运行的十个累积阶段，其演进关系如下：

阶段	Commit	开发目标
00	4cd3e021	正确性基线
01	c632ac33	folio 与只读文件页直映
02	a2f3320d	多核运行稳定化
03	56505b0d	PCP 页缓存
04	d84d85db	mprotect 与批量 TLB
05	7d3da780	dentry 与 inode 并发
06	3a7d6374	异步 I/O 与 ext4 锁外 I/O
07	8608d4e1	fault-around 与 COW/TLB 修复
08	7d4e6be8	文件系统与锁竞争综合优化
09	8fd4e55f	main 最终稳定版本

表 5-1　BuildStorm 开发阶段与代码提交

审核者可以用 Git 逐步查看任一阶段的提交内容和相邻差异：

```
git show 56505b0d
git diff 56505b0d..d84d85db
```

以 `mprotect` 为例：前序缺页优化后，路径采样显示热点转移到权限修改；比较上述提交可以看到 per-VMA 定位和批量 TLB 刷新实现。`profiles/stage-03/wait/mprotect.svg` 与 `stage-04` 对应图记录了累计延迟从 1.907 秒降至 0.468 秒。其他阶段也可按“第 2 章热点—相邻提交差异—第 4 章结果”复核。

5.2.3　复现正式实验与图表

完整实验材料位于 `docs/buildstorm-report/experiments/la-20260813/`。运行需要 `sdcard-la.img`、仓库指定的 Rust/LoongArch 工具链和实验目录使用的 QEMU 10.0.2；具体路径与校验值见 `manifest.json`。在仓库根目录执行：

```
bash docs/buildstorm-report/experiments/la-20260813/scripts/build_kernels.sh
python3 docs/buildstorm-report/experiments/la-20260813/scripts/run_all.py \
  --repo "$(pwd)" --cooldown 30 --max-attempts 3
python3 docs/buildstorm-report/experiments/la-20260813/scripts/summarize.py
```

构建脚本通过 `git archive` 生成各提交的干净快照并应用 BuildStorm-only 补丁。Linux 和十个 Moss 阶段各串行运行三次，失败自动留档并重试，中断后可继续。原始数据、汇总和图表分别位于 `stages/`、`runs.csv`、`stages.csv` 与 `charts/`。

CPU 与等待火焰图必须独立采集，不能计入正式耗时。首末阶段 CPU 图可用 `scripts/run_profile.py` 采集；mutex 与 `mprotect` 图按以下命令复现：

```
bash docs/buildstorm-report/experiments/la-20260813/scripts/
build_wait_profile_kernels.sh
python3 docs/buildstorm-report/experiments/la-20260813/scripts/run_wait_profiles.py
python3 docs/buildstorm-report/experiments/la-20260813/scripts/render_wait_profiles.py
```

正式样本只有在工具链与 minibuild 通过、`BUILDSTORM_COMPILE` 报告 `ok=true`/LoongArch/8 核、产物大小正确、无 panic 且正常结束时才进入统计。上述提交、原始日志、脚本与图表共同构成了从问题定位到最终验证的可追溯开发记录。

第 6 章 总结与展望

本文档完整记录了 MossOS 针对 BuildStorm 测例的问题定位、优化设计与实验验证过程。核心结论如下：

- 瓶颈定位明确：LoongArch 基线火焰图中 ext4 buffer-cache 形成高度集中的热塔，三次正式样本平均仅使用 3.077 核，6～8 核区间占比只有 0.20%，与缺页、页缓存和 ext4 路径上的锁嵌套判断相互印证；
- 优化方向正确：六项核心优化按依赖顺序实施，逐一拆除“文件缺页 → 物理页分配 → 页缓存 → ext4 → 块设备 → mprotect”链路的锁嵌套和外层大临界区，后续优化继续缩短回收与 TLB 同步路径；
- 端到端效果显著：LoongArch 同口径三次测量的中位数从 993.08 秒降至 354.27 秒（-64.33%，2.80 倍加速），比同机 Linux 基线的 413.31 秒快 14.28%；6～8 核占比从 0.20% 升至 35.98%；
- 证据分级透明：正式结果均保留三次样本和最小—最大范围，profiling 与计时严格分开；阶段 02 和阶段 06 的小幅回退同样如实呈现，不把累积阶段结果误写成单项优化的严格 A/B 因果证据。

在性能优化层面，当前工作仍有若干值得继续深入的方面。

- 参数自适应：现有优化主要围绕 BuildStorm 的读密集、顺序访问特征展开，folio 的 order、fault-around 的窗口大小和 PCP 的 batch 参数均根据该场景人工设定。后续可以结合访问模式与内存压力动态调整这些参数，使其更好地适应写密集或随机 I/O 负载；
- 同步开销拆解：AddrSpace 三段式为并发缺页提供了基础框架，但 COW commit 引入的远程 TLB flush 等同步成本，是否已经被 UnmapGather 和 active-root TLB 的收益完全抵消，仍需进一步定量分析；
- **mprotect** 并发性：当前优化将 `mprotect` 的处理范围从全局扫描缩小到目标 VMA，但它仍持有 AddrSpace 写锁，并与 `mmap`、`munmap` 串行。也就是说，锁内工作量已经从 `O(N_global)` 降至 `O(K_local)`，锁本身的竞争仍有继续优化的空间；
- **ext4** 元数据并发：`create`、`unlink` 和 `rename` 等元数据操作仍可进一步细化锁粒度。在大规模并行构建产生大量临时文件时，这些路径可能成为新的瓶颈；